INTERNSHIP REPORT

A report submitted in partial fulfillment of the requirements for the Award of Degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

By

P. YUGENDRA

Regd. No.: 239Y1A05C2

Under Supervision of

BHARAT CARE LIFE LINE FOUNDATION

in association with ZERO P4 POVERTY

Project Title: Customer Churn Prediction Using Machine Learning

(Duration: 45-Day Internship, Summer 2026)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

K.S.R.M. COLLEGE OF ENGINEERING

(An Autonomous Institution)

Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu

Accredited by NAAC with A+ Grade & B.Tech (EEE, ECE, CSE, CE and ME) Programs by NBA

KADAPA, ANDHRA PRADESH

2026 – 2027

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
K.S.R.M. COLLEGE OF ENGINEERING
(An Autonomous Institution)
KADAPA

CERTIFICATE

This is to certify that the Internship Report entitled **"Customer Churn Prediction Using Machine Learning"** submitted by **P. Yugendra** (Regd. No.: 239Y1A05C2) is a record of work carried out during the **45-Day Data Science Internship** conducted under the **AVISHKARANA ANDHRA SUMMER INTERNSHIP** by **BHARAT CARE LIFE LINE FOUNDATION**, in association with **ZERO P4 Poverty**, from the summer of 2026, and submitted during the **2026 – 2027** academic year, in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in **COMPUTER SCIENCE AND ENGINEERING** at **K.S.R.M. COLLEGE OF ENGINEERING, KADAPA**.

The internship was completed under the guidance of **A. Vamsi Reddy**, Coordinator, Bharat Care Life Line Foundation. The certificate of achievement was issued on **15 August 2026**.

Internship Coordinator	**External Examiners**	**Head of the Department**
A. Vamsi Reddy		Dr. P. V. Kishore
Coordinator, Bharat Care Life Line Foundation		Associate Professor & HoD, CSE

CERTIFICATION

The following certificate of achievement was issued to the candidate by **BHARAT CARE LIFE LINE FOUNDATION** on **15 August 2026** for the successful completion of the 45-Day Internship in Data Science under the Avishkarana Andhra Summer Internship programme.



Certificate of Achievement – 45-Day Internship in Data Science

The certificate is reproduced here as documentary evidence of the internship. The original document is available for verification.

K.S.R.M. COLLEGE OF ENGINEERING

(An Autonomous Institution Affiliated to JNTUA, Ananthapuramu)
Kadapa, Andhra Pradesh, India – 516 005

VISION

To emerge as a centre of excellence in technical education and research, nurturing psychologically strong, socially responsible and globally competent professionals for sustainable development.

MISSION

M1: To provide multidisciplinary education through enriched curricula, emerging technologies and effective pedagogical practices.

M2: To foster research, innovation and entrepreneurship through industry–academia linkage.

M3: To instill leadership skills and ethical values among professionals through community engagement, contributing to the development of society and the nation.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision

To emerge as a centre of excellence in Computer Science and allied fields, producing technically competent professionals imbued with ethical values who drive sustainable digital transformation.

Mission

M1: To empower learners through student-centred teaching, cutting-edge technologies, and a curriculum aligned with industry needs.

M2: To nurture a culture of research, innovation, and entrepreneurship, and to strengthen industry–academia partnerships that address real-world and societal challenges.

M3: To produce technically competent graduates with strong ethical values, a sense of social responsibility, and leadership capabilities, who continue to learn and contribute to sustainable digital transformation.

ACKNOWLEDGEMENT

First and foremost, I would like to express my sincere gratitude to **BHARAT CARE LIFE LINE FOUNDATION** and the **AVISHKARANA ANDHRA SUMMER INTERNSHIP** for providing me with the opportunity to undertake a 45-day internship in Data Science and to gain practical experience through the project titled **"Customer Churn Prediction Using Machine Learning."**

I am thankful to the ZERO P4 Poverty initiative for supporting the programme and for creating an environment in which students can apply classroom knowledge to real social and business problems.

I am deeply grateful to **A. Vamsi Reddy**, Coordinator, Bharat Care Life Line Foundation, for his constant guidance, patient reviews and constructive feedback throughout the internship. His suggestions shaped both the analytical work and the final report.

I am extremely thankful to our beloved Managing Director and Vice Chairman, **Dr. K. Chandra Obul Reddy Garu**, for his encouragement and support.

I am highly indebted to our Principal, **Dr. T. Nageswara Prasad**, for providing the necessary facilities and support to accomplish this internship.

I would like to express my sincere thanks to **Dr. P. V. Kishore**, Associate Professor and Head of the Department of CSE, for his constant encouragement and support throughout my academic work.

I am also grateful to **Dr. V. Lokeswara Reddy**, Professor and Internship Coordinator, Department of CSE, for his valuable guidance in obtaining and completing this internship.

I extend my sincere thanks to all the faculty members of the Department of Computer Science and Engineering, my friends and my family for their encouragement and help throughout the internship.

Finally, I would like to thank everyone who directly or indirectly contributed to the successful completion of this internship.

Name: P. Yugendra

Regd. No.: 239Y1A05C2

ABSTRACT

Customer churn is the loss of customers who stop using a company's product or service. In subscription-based industries such as telecommunications, churn directly reduces recurring revenue and increases the cost of replacing lost customers. Retention teams, however, cannot contact every customer. They need a ranked list of who is most likely to leave, an explanation of why, and a practical way to test retention options.

This project develops a complete machine learning system for customer churn prediction using the publicly available IBM Telco Customer Churn dataset, which contains 7,043 customers and 21 attributes covering demographics, subscribed services, contract details, billing and tenure. The data is cleaned, explored and enriched with three engineered features: the number of active services, average monthly spend and a new-customer flag. Three classification models—Logistic Regression, Random Forest and Histogram Gradient Boosting—are compared using stratified five-fold cross-validation.

Logistic Regression achieves the best cross-validated ROC-AUC (0.847) and is selected as the final model. On a held-out test set of 1,409 customers, it reaches a ROC-AUC of 0.846, a recall of 79.4%, a precision of 50.6% and an F1 score of 0.618. In practical terms, the model catches 297 of the 374 customers who actually churn. The results also show that contract type and tenure are the strongest signals: month-to-month customers churn at 42.7%, compared with 2.8% on two-year contracts.

The selected model is deployed in an interactive Streamlit application. The application shows a customer's churn probability, explains the factors behind the score, and re-scores the customer under alternative contract, payment and service scenarios. The project demonstrates a reproducible end-to-end data science workflow—from data preparation and model selection to evaluation, interpretation and deployment—and provides a practical decision-support tool for retention planning.

Keywords: *customer churn; telecommunications; machine learning; logistic regression; ROC-AUC; recall; Streamlit; retention analytics.*

Programmes and Opportunities

The **Avishkarana Andhra Summer Internship** is a structured summer programme that connects undergraduate students with industry and social-sector organisations for mentored, project-based learning. It gives students an opportunity to work on real problems, apply technical skills in a professional setting and understand how organisations use data to make decisions.

The 45-day Data Science internship was hosted by **Bharat Care Life Line Foundation** in association with **ZERO P4 Poverty**. The programme combined independent project work with periodic reviews, giving the intern experience in data preparation, analysis, machine learning and communication of results.

Organization Information

Bharat Care Life Line Foundation is a social-purpose organisation that works on community welfare, skills development and livelihood initiatives. Through its collaboration with **ZERO P4 Poverty**, it supports programmes that use education, technology and enterprise to reduce poverty and expand opportunity.

As part of the Avishkarana Andhra Summer Internship, the foundation provided the internship framework, mentorship and project review. The Data Science project described in this report was completed within that framework.

Internship at a Glance

Item	Details
Programme	Avishkarana Andhra Summer Internship
Host organisation	Bharat Care Life Line Foundation
In association with	ZERO P4 Poverty
Domain	Data Science
Duration	45 days (Summer 2026)
Project	Customer Churn Prediction Using Machine Learning
Coordinator	A. Vamsi Reddy
Certificate issued	15 August 2026

Table F.1: Internship details

Benefits to the Institution and the Host Organisation

The project offers practical value to both the host organisation and the institution. For the host organisation, it demonstrates how routinely collected customer data can be converted into a ranked retention list, an explanation of risk drivers and a simple what-if tool for planning retention offers. The same workflow can be adapted to other programmes that need to identify participants or beneficiaries at risk of disengagement.

For the institution, the report documents a complete applied data science workflow that can serve as a reference for future internships and student projects. It shows how statistical reasoning, software engineering and communication are combined in a real deliverable rather than an isolated classroom exercise.

Learning Objectives

- To gain practical experience in Python programming and data analysis.
- To clean, validate and prepare a real-world dataset for machine learning.
- To perform exploratory data analysis and communicate the findings clearly.
- To engineer interpretable features from raw customer and account data.
- To train, compare and validate classification models using scikit-learn.
- To evaluate imbalanced classification models with appropriate metrics.
- To interpret model behaviour and connect it to business decisions.
- To deploy a trained model in an interactive Streamlit application.
- To document the work professionally and present it to a review panel.

Expected Outcomes

At the end of the internship, the candidate is expected to demonstrate a reproducible machine learning pipeline, a validated model with honest performance estimates, a working demonstration application, and a written report that explains the design choices, results and limitations of the work.

INDEX

S. No.	CONTENTS	PAGE
1.	Introduction	10
2.	History	15
3.	Definition	18
4.	System Architecture	22
5.	Applications	26
6.	Technologies Used	29
7.	Advantages and Disadvantages	32
8.	Requirements	36
9.	Methodology and Source Code	39
10.	Results and Output Screenshots	47
11.	Conclusion	60
12.	Bibliography and References	63

Note: page numbers refer to the printed page numbers at the bottom of each page. The index is updated after the final pagination of the report.

1. INTRODUCTION

Telecommunications is one of the most competitive subscription industries in the world. Customers can compare tariffs in minutes, switch providers without visiting a store, and bundle or unbundle services as their needs change. In this environment, acquiring a new customer is considerably more expensive than retaining an existing one, and a single percentage point of avoidable churn can translate into a large amount of recurring revenue. Retention is therefore no longer only a customer-service concern; it is a core commercial capability that depends on the ability to anticipate which customers are about to leave and to act before they do.

This report documents the design, implementation and evaluation of a **Customer Churn Prediction** system completed during a 45-day Data Science internship under the **AVISHKARANA ANDHRA SUMMER INTERNSHIP**, hosted by **BHARAT CARE LIFE LINE FOUNDATION** in association with **ZERO P4 Poverty**. The system uses the publicly available IBM Telco Customer Churn dataset, compares three classification algorithms, selects the most suitable model on cross-validated performance, and exposes the result through an interactive Streamlit application that a retention team can use without writing code.

1.1 Background and Motivation

Every subscription business maintains a record of its relationship with each customer: how long the customer has been with the company, which services are active, how much is paid each month, which contract has been signed and how the bill is settled. Individually these fields are operational data. Collectively they describe behaviour, and behaviour carries signal. A customer on a month-to-month contract who pays by electronic check, has been with the company for three months and has never contacted technical support is not a random customer; that combination is statistically much more likely to end in cancellation than a two-year customer with automatic payment and a long tenure.

Traditional reporting answers the question *what happened*. A churn dashboard that shows last month's cancellations is useful for accounting, but it cannot prevent the next cancellation. Predictive analytics changes the question from *what happened* to *what is likely to happen*, and then to *what should we do about it*. A supervised machine learning model learns the relationship between customer attributes and the observed churn outcome, and applies that relationship to

current customers. The output is a probability, not a verdict. Customers can then be ranked, and retention resources can be concentrated on the segment where the expected benefit is highest.

The motivation for this project is practical. Retention teams usually have limited capacity: a call centre can contact only a few hundred customers per week, and every retention offer has a cost. A model that identifies the highest-risk customers with reasonable precision allows the team to spend that capacity where it matters. An explanation of why each customer is at risk allows the team to choose an appropriate intervention - a contract upgrade, a service review, a payment-method change or a goodwill credit - instead of applying the same discount to everyone.

1.2 Problem Statement

The host organisation and its partners work with communities and enterprises that depend on sustained participation. In a commercial subscription setting, the problem can be stated precisely: given a customer's demographic, account, service and billing attributes at a point in time, estimate the probability that the customer will cancel the service in the near future, and present that estimate in a form that supports a retention decision.

The project addresses four specific sub-problems:

1. **Data preparation.** The raw dataset contains a numeric field stored as text, missing values and an imbalanced target. The data must be cleaned and transformed without leaking information from the test set.
2. **Model selection.** Several algorithms can model a binary outcome. The project must compare them under a fair protocol and choose a model that performs well on customers it has never seen.
3. **Evaluation.** Accuracy is misleading when only about one in four customers churns. The evaluation must focus on ranking quality (ROC-AUC) and on the proportion of churners that are actually caught (recall), while reporting precision and F1 as trade-off indicators.
4. **Deployment and explanation.** A model that exists only in a notebook has limited value. The result must be packaged into an application that accepts customer details, returns a risk score, explains the main contributing factors and allows a what-if analysis of retention scenarios.

1.3 Objectives

The objectives of the internship project are as follows:

- To understand the business meaning of customer churn and the cost structure that makes retention economically important.
- To perform a structured data-quality audit and clean the Telco Customer Churn dataset reproducibly.
- To explore the data visually and quantify the relationship between contract, tenure, charges, services and churn.
- To engineer a small number of interpretable features that capture service breadth, spending behaviour and customer maturity.
- To build a leakage-free preprocessing and modelling pipeline using scikit-learn.
- To train and compare Logistic Regression, Random Forest and Gradient Boosting with stratified cross-validation.
- To evaluate the selected model on a held-out test set using accuracy, precision, recall, F1, ROC-AUC and PR-AUC.
- To interpret the model using permutation importance and connect the results to retention practice.
- To deploy the model in an interactive Streamlit application with a risk score, factor explanation and what-if simulator.
- To document the work in a professional internship report and present it to an academic and industry audience.

1.4 Scope and Delimitations

The scope of the project is a single-tabular, binary classification problem using historical customer records. The model predicts whether a customer will churn, not when the customer will churn. It does not use text, call transcripts, network telemetry, social media activity or competitor pricing, because those sources are not part of the public dataset. The model is intended as a decision-support tool; it does not automate cancellations, pricing or contract decisions.

The dataset is a cross-sectional snapshot rather than a longitudinal event log. As a result, the model cannot capture the effect of a retention campaign that has already been run, nor can it account for seasonality. These limitations are discussed in Chapter 11.

1.5 Dataset Overview

The IBM Telco Customer Churn dataset contains **7,043 customers** and **21 columns**. Each row represents one customer at the time the snapshot was taken. The target column, `Churn`, records whether the customer left. The dataset is imbalanced: **1,869 customers (26.5%) churned**, while 5,174 (73.5%) stayed. The columns can be grouped as shown in Table 1.1.

Group	Attributes	Type	Example
Identity	customerID	Text	7590-VHVEG
Demographics	gender, SeniorCitizen, Partner, Dependents	Categorical / binary	Female, 0, Yes, No
Account	tenure, Contract, PaperlessBilling, PaymentMethod	Numeric / categorical	1 month, Month-to-month, Yes, Electronic check
Services	PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies	Categorical	Yes, No, Fiber optic
Billing	MonthlyCharges, TotalCharges	Numeric	29.85, 29.85
Target	Churn	Binary	Yes / No

Table 1.1: Structure of the Telco Customer Churn dataset

The dataset is widely used in churn research and teaching because it combines demographic, contractual and behavioural variables in a realistic proportions. It is small enough to train on a laptop, yet rich enough to demonstrate a complete machine learning workflow. Chapter 9 describes the audit and preparation of the data in detail.

1.6 Contributions of the Project

The project makes the following practical contributions:

1. A reproducible data-cleaning and feature-engineering pipeline that can be re-run from the raw CSV with a single command.
2. An honest model comparison using stratified five-fold cross-validation and a held-out test set that is never used for model selection.
3. A selected model with a documented operating threshold and an explicit discussion of the precision-recall trade-off.
4. A permutation-importance analysis that identifies the variables a retention team should monitor.
5. A deployed Streamlit application with a live risk score, a factor explanation, a what-if simulator, a model-performance view and a data-insights view.

6. A written record of the methodology, results and limitations suitable for academic review and for handover to a future intern.

1.7 Organisation of the Report

The remainder of the report is organised as follows. **Chapter 2** traces the history of churn analysis and the technologies used. **Chapter 3** defines churn and the machine learning concepts that the project relies on. **Chapter 4** presents the system architecture. **Chapter 5** discusses applications of the solution. **Chapter 6** describes the technologies used. **Chapter 7** lists advantages and disadvantages. **Chapter 8** states the hardware, software and data requirements. **Chapter 9** explains the methodology and includes selected source code. **Chapter 10** presents the results and output screenshots. **Chapter 11** concludes the report and outlines future work. **Chapter 12** lists the references consulted.

2. HISTORY

Churn analysis did not begin with machine learning. It emerged from the commercial pressures of deregulated telecommunications markets, matured through the data-mining wave of the 1990s, and was transformed by the open-source scientific computing stack in the 2010s. Understanding this history explains why churn models look the way they do today and why certain metrics, particularly recall and ROC-AUC, have become standard in retention analytics.

2.1 The Emergence of Customer Retention in Telecommunications

For much of the twentieth century, telephone service was delivered by state-owned or regulated monopolies. Customers had little choice, and churn was a minor operational issue. The deregulation of telecommunications in the United States in the 1980s, followed by similar reforms in Europe and Asia, created competitive markets with multiple long-distance and mobile carriers. Switching became possible, tariffs became visible, and acquisition costs rose sharply. Carriers responded by building customer databases and by tracking cancellation rates as a key performance indicator.

Early retention practice was largely reactive and rule-based. A customer who had not renewed a contract was contacted; a customer who complained repeatedly was escalated. As billing systems became computerised in the 1980s and 1990s, carriers accumulated detailed records of usage, payment and service calls. This data made it possible to move from intuition to statistical segmentation, and the first churn scorecards were built using logistic regression on a small number of account variables.

2.2 From Descriptive Reporting to Predictive Modelling

The 1990s saw the emergence of data mining as a discipline. Decision trees, neural networks and logistic regression were applied to customer databases in banking, insurance and telecommunications. The landmark work of Berry and Linoff on data mining techniques, and the development of the CRISP-DM process model in 1996, gave practitioners a shared vocabulary for business understanding, data preparation, modelling, evaluation and deployment. Churn prediction became a standard case study in both academic research and commercial analytics.

A second wave arrived with ensemble methods. Random Forests, introduced by Leo Breiman in 2001, combined many decision trees to reduce variance and improved predictive accuracy on tabular data. Gradient boosting machines, developed by Jerome Friedman in the same period, built trees sequentially to correct previous errors and became the dominant approach in many competitions. These methods are now available in mature open-source libraries such as scikit-learn, XGBoost and LightGBM, which brought high-quality modelling to ordinary analysts.

Despite the sophistication of modern algorithms, logistic regression remains a strong baseline for churn. It is fast, stable, interpretable and often competitive when the relationship between features and the log-odds of churn is approximately linear. This project treats it as a serious candidate rather than a token baseline, and the results in Chapter 10 show why.

2.3 The Rise of the Open-Source Data Science Stack

Python was created by Guido van Rossum in 1991 as a general-purpose language emphasising readability. Its adoption for scientific computing accelerated after NumPy (2006) and pandas (2008) provided efficient arrays and data frames. scikit-learn, released in 2007 by David Cournapeau and later maintained by a broad community, unified model selection, preprocessing and evaluation under a consistent API. Matplotlib and Seaborn made publication-quality visualisation accessible, while Jupyter notebooks changed the way analysts explored data.

Streamlit, launched in 2019, lets a Python script become an interactive web application without front-end code, lowering the barrier between analysis and use.

2.4 History of the Telco Customer Churn Dataset

The dataset used in this project was published by IBM as a sample dataset for analytics education. It represents a fictional telecommunications company with 7,043 customers in California and contains demographic information, subscribed services, contract details, billing information and a churn label. Because it is synthetic but realistic, it has become one of the most widely used benchmarks for binary classification with mixed numerical and categorical features.

The dataset's popularity has both benefits and risks. The benefit is comparability: many published analyses provide context for the results obtained here. The risk is overfitting to a well-

known benchmark. For that reason, this project emphasises methodological discipline - a fixed train/test split, cross-validation on the training set only, and a clear separation between model selection and final evaluation - rather than chasing a leaderboard score.

2.5 The Internship Context

The project was carried out during a **45-day Data Science internship** under the **AVISHKARANA ANDHRA SUMMER INTERNSHIP**, hosted by **BHARAT CARE LIFE LINE FOUNDATION** in association with **ZERO P4 Poverty**. The programme's structure - an independent project with periodic reviews - mirrors professional analytics work: the intern defines the problem, prepares the data, builds and validates the model, and communicates the result to a non-technical audience. The certificate of achievement was issued on **15 August 2026** and is reproduced in the front matter of this report.

The choice of a churn project was deliberate. Churn is a problem that connects statistics, software engineering and business communication. It requires the analyst to reason about class imbalance and evaluation metrics, to write clean and reproducible code, and to explain a probability in language that a retention manager can act on. These are precisely the competencies that the internship aimed to develop.

3. DEFINITION

This chapter defines the central concepts used throughout the report. It begins with the business definition of churn, distinguishes its different forms, and then translates the business problem into a machine learning formulation. The evaluation metrics are defined mathematically so that the results in Chapter 10 can be read without ambiguity.

3.1 Customer Churn

Customer churn, also called customer attrition, is the loss of a customer who stops using a company's product or service. In a contractual setting, churn is observed when a subscription is cancelled or not renewed. In a non-contractual setting, such as retail banking or e-commerce, churn is inferred from a prolonged absence of activity. The Telco dataset is contractual: the `Churn` column explicitly records whether the customer left the company.

Churn is usually expressed as a rate over a defined period. The overall churn rate in the dataset is 26.5%, calculated as the number of customers who left divided by the total number of customers. This aggregate figure conceals large differences between segments: the churn rate is 42.7% for month-to-month customers but only 2.8% for customers on two-year contracts. Segment-level rates are therefore more useful for management than a single company-wide number.

3.2 Types of Churn

Retention analysts distinguish several forms of churn:

- **Voluntary churn.** The customer actively cancels or does not renew. This is the form recorded in the Telco dataset and the form that retention campaigns can influence.
- **Involuntary churn.** The customer is lost because of payment failure, fraud or credit action rather than a deliberate decision. It is often addressed through billing recovery rather than predictive modelling.
- **Partial churn.** The customer remains but reduces usage or drops some services. It is an early warning signal and often precedes full churn.
- **Silent churn.** The customer remains formally subscribed but stops engaging. In non-contractual businesses this is the dominant form and requires an activity-based definition.
- **Seasonal churn.** Cancellations concentrated in a particular period, such as the end of an academic year or a promotional cycle. It should be separated from structural churn.

The project models voluntary churn at the customer level. This is the most actionable definition for a telecom operator because the customer's decision is preceded by observable account and service characteristics.

3.3 Key Terms

Term	Definition
Churn	Loss of a customer who stops using the product or service.
Churn rate	Number of customers lost in a period divided by the customers at risk at the start of the period.
Feature	An input variable used by the model, such as tenure or contract type.
Target	The outcome the model learns to predict; here, the binary churn label.
Class imbalance	A situation in which one outcome is much more frequent than the other.
Training set	The portion of data used to fit the model parameters.
Validation set	Data used to compare models and tune settings without touching the test set.
Test set	A held-out portion used once to estimate final performance.
Pipeline	A sequence of preprocessing and modelling steps applied as one object.
Probability	A number between 0 and 1 expressing the model's estimated likelihood of churn.
Threshold	The probability cut-off above which a customer is classified as a churner.
ROC-AUC	The probability that the model ranks a random churner above a random non-churner.
Recall	The proportion of actual churners that the model identifies.
Precision	The proportion of customers flagged as churners who actually churn.
Permutation importance	The drop in model performance when a feature's values are randomly shuffled.

Table 3.1: Glossary of terms used in the report

3.4 Churn as a Binary Classification Problem

Let each customer be represented by a feature vector **x** containing account, service and billing attributes, and let **y** be a binary label with $y = 1$ for churn and $y = 0$ for retention. A classifier estimates the conditional probability $P(y = 1 \mid x)$. The model is trained on a labelled sample and then applied to new customers. A decision rule converts the probability into a class label: if the estimated probability exceeds a threshold t, the customer is classified as a predicted churner.

The choice of threshold is a business decision, not a statistical one. A low threshold flags more customers, catching more true churners at the cost of contacting more customers who would have stayed. A high threshold conserves retention capacity but misses customers who are about to leave. This project uses the conventional 0.50 threshold for the reported results and discusses the trade-off explicitly in Chapter 10. The application also reports the raw probability, so a manager can apply a different threshold if the economics of the campaign justify it.

The features are a mixture of numeric variables (tenure, monthly charges, total charges) and categorical variables (contract, payment method, internet service). Categorical variables cannot be used directly by most algorithms and must be encoded. Numeric variables have different scales and must be standardised for logistic regression. These transformations are learned from the training data and applied unchanged to the test data, which prevents information leakage.

3.5 Evaluation Metrics

A binary classifier produces four outcomes: true positives (TP), customers who churned and were correctly flagged; true negatives (TN), customers who stayed and were correctly left alone; false positives (FP), customers who stayed but were flagged; and false negatives (FN), customers who churned but were missed. The metrics used in this report are defined in Table 3.2.

Metric	Formula	Interpretation
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Overall proportion of correct predictions; misleading under class imbalance.
Precision	TP / (TP + FP)	Of the customers flagged, the proportion who actually churn.
Recall	TP / (TP + FN)	Of the customers who churn, the proportion that is caught.
F1 score	2 × (Precision × Recall) / (Precision + Recall)	Harmonic mean of precision and recall.
ROC-AUC	Area under the true-positive vs false-positive curve	Probability of ranking a churner above a non-churner.
PR-AUC	Area under the precision-recall curve	Summary of precision-recall performance; sensitive to imbalance.

Table 3.2: Evaluation metrics and their interpretation

Accuracy is included for completeness but is not used to select the model. A trivial classifier that predicts *no churn* for every customer would achieve 73.5% accuracy while catching zero churners. The project therefore selects on cross-validated ROC-AUC, which measures ranking quality independent of the threshold, and reports recall and precision at the chosen operating point.

ROC-AUC has a precise interpretation: it is the probability that a randomly chosen churner receives a higher risk score than a randomly chosen non-churner. A value of 0.5 corresponds to random ranking and 1.0 to perfect ranking. The selected model achieves 0.846, which means that in roughly 85 of every 100 randomly formed churner/non-churner pairs, the churner is ranked higher. This is a strong result for tabular customer data.

3.6 Business Interpretation of a Probability

A churn probability is not a certainty about an individual. It is an estimate of the frequency with which customers with similar characteristics leave. If the model assigns a probability of 0.72 to a customer, the correct interpretation is that among customers with a similar profile, approximately 72% churned in the historical data. The retention team can use that estimate to prioritise, but it should not treat the score as a guarantee.

This distinction matters for communication. Saying *this customer will leave* is overconfident and can damage trust when the customer stays. Saying *this customer has a high estimated risk and here are the factors driving it* is accurate, actionable and consistent with the model's actual output. The Streamlit application is designed around this interpretation: it presents a risk band, a probability and a list of contributing factors rather than a binary verdict.

4. SYSTEM ARCHITECTURE

The system is organised as a sequence of well-defined stages that transform raw customer records into an actionable risk score. The architecture separates offline model development from online inference, keeps preprocessing inside the saved pipeline, and exposes the model through a thin presentation layer. This separation makes the system reproducible, testable and easy to extend.

4.1 Architectural Overview

Figure 4.1 shows the end-to-end architecture. Raw data enters the system once and is cleaned and enriched before being split into training and test partitions. The training partition is used for cross-validation and model selection; the test partition is held back until the final evaluation. The selected pipeline, together with its metadata and evaluation artifacts, is persisted to disk. The Streamlit application loads the persisted pipeline and serves predictions to the user.

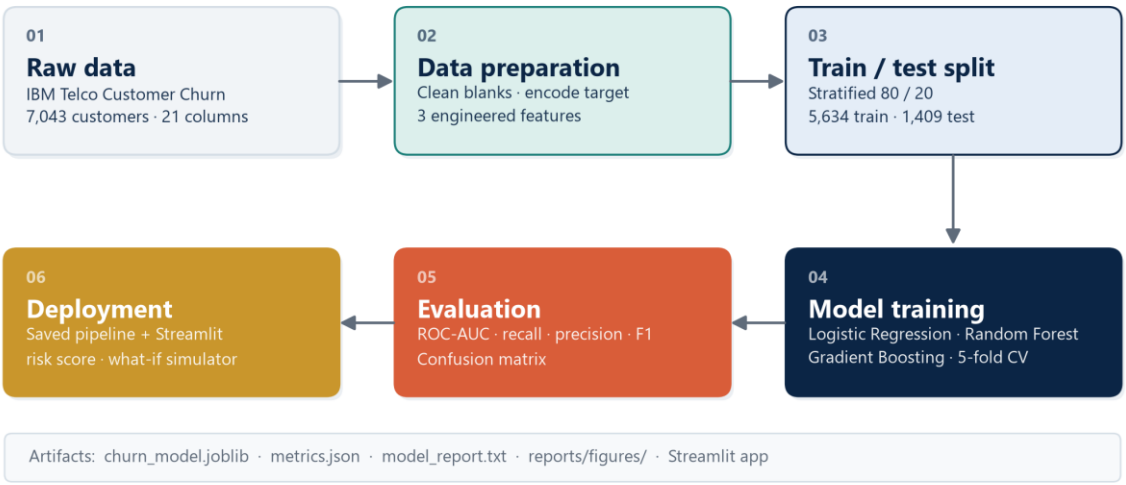


Figure 4.1: End-to-end system architecture

The architecture is deliberately batch-oriented on the training side and request-oriented on the serving side. Training is an offline, reproducible job that can be re-run when new data arrives. Serving is a stateless function of the customer record and the saved pipeline, which makes the application easy to host and reason about.

4.2 Architectural Principles

Five principles guided the design:

1. **Single source of truth.** All preprocessing is defined inside the scikit-learn pipeline. The same transformations are used during training, evaluation and inference, eliminating training-serving skew.
2. **No leakage.** The test set is created once with a fixed random seed and is never used for feature selection, hyperparameter tuning or threshold selection.
3. **Reproducibility.** Random seeds, package versions, input data and output artifacts are recorded so that the experiment can be reconstructed.
4. **Separation of concerns.** Data preparation, modelling, evaluation and presentation are implemented in separate modules with clear responsibilities.
5. **Explainability by design.** The system stores feature names and permutation importance so that predictions can be explained to a non-technical audience.

4.3 System Components

Component	Responsibility	Implementation
Data layer	Stores the raw CSV and the cleaned modelling table.	data/Telco-Customer-Churn.csv
Preparation module	Loads, validates, cleans and enriches the data.	src/train.py
Preprocessing pipeline	Imputes, scales and one-hot encodes features.	scikit-learn ColumnTransformer
Model layer	Trains and compares three classifiers; selects the best.	LogisticRegression, RandomForestClassifier, HistGradientBoostingClassifier
Evaluation layer	Computes metrics, curves and permutation importance.	scikit-learn metrics and inspection
Artifact layer	Persists the pipeline, metrics and human-readable report.	joblib, JSON, text
Presentation layer	Provides risk scoring, explanations and what-if analysis.	Streamlit application (app.py)
Documentation layer	Records methodology, results and operating instructions.	Markdown, DOCX and PDF reports

Table 4.1: System components and responsibilities

4.4 End-to-End Workflow

The workflow consists of eight stages:

1. **Acquisition.** The raw CSV is read into a pandas DataFrame. The schema is checked and the target is mapped from Yes/No to 1/0.
2. **Cleaning.** `TotalCharges` is converted from text to numeric. Eleven blank values, all belonging to customers with zero tenure, are imputed from `MonthlyCharges`. The identifier column is dropped.

3. **Feature engineering.** Three features are added: the number of active services, the average monthly spend and a new-customer indicator.
4. **Splitting.** The data is divided into 80% training and 20% test partitions using stratified sampling with a fixed random seed.
5. **Preprocessing.** Numeric features are median-imputed and standardised; categorical features are most-frequent-imputed and one-hot encoded with unknown-category tolerance.
6. **Training and selection.** Three models are evaluated with stratified five-fold cross-validation on the training partition. The model with the highest mean cross-validated ROC-AUC is selected.
7. **Evaluation.** The selected model is fitted on the full training partition and evaluated once on the test partition. Confusion matrix, classification report, ROC curve, PR curve and permutation importance are computed.
8. **Packaging and deployment.** The fitted pipeline and metadata are saved with joblib. The Streamlit application loads them and serves the interactive interface.

4.5 Data Flow and Transformations

Table 4.2 summarises how the data changes as it moves through the pipeline. The number of columns increases from 21 to 23 after feature engineering, and then expands substantially after one-hot encoding. The exact encoded width depends on the categories observed in the training data, which is why the encoder is fitted inside the pipeline rather than on the full dataset.

Stage	Rows	Columns	Transformation
Raw CSV	7,043	21	As downloaded; TotalCharges stored as text.
Cleaned	7,043	20	Target encoded; identifier dropped; TotalCharges numeric.
Feature-engineered	7,043	23	NumServices, AvgMonthlySpend and IsNewCustomer added.
Training partition	5,634	22 features	Stratified 80% split; target separated.
Test partition	1,409	22 features	Held out until final evaluation.
Encoded matrix	5,634	≈ 45	Numeric scaled; categorical variables one-hot encoded.

Table 4.2: Data transformations through the pipeline

4.6 Deployment Architecture

The deployed application is a single-process Streamlit server. On start-up it loads `models/churn_model.joblib`, which contains the fitted pipeline, the ordered feature list and the model name. It also loads `reports/metrics.json` for the performance view. When the user

changes an input in the sidebar, Streamlit re-executes the script, builds a one-row DataFrame with the engineered features, calls `predict_proba` and renders the result.

This architecture is appropriate for demonstration and internal review. It requires no database, no authentication layer and no separate API server. For production use, the same pipeline could be wrapped in a REST endpoint and placed behind an authenticated gateway; the model artifact would not need to change. Chapter 11 discusses this path.

4.7 Design Decisions and Trade-offs

Several design decisions shaped the system:

- **Pipeline over ad-hoc preprocessing.** Wrapping all transformations in a `ColumnTransformer` and `Pipeline` costs a little extra code but guarantees that inference uses exactly the same transformations as training.
- **Logistic Regression as a serious candidate.** The model is often treated as a baseline, but it offers interpretability, fast training and stable probabilities. It won the cross-validated comparison and was therefore selected on evidence rather than assumption.
- **Fixed 0.50 threshold for reporting.** Reporting at a standard threshold makes the results comparable with other studies. The application exposes the probability so that a different operating point can be chosen for a specific campaign.
- **Permutation importance over coefficients.** Coefficients are informative for logistic regression but depend on the encoding and scale. Permutation importance measures the actual drop in ROC-AUC when a feature is disrupted, which is easier to communicate.
- **Streamlit over a custom web application.** Streamlit delivers an interactive experience in a single Python file, which is appropriate for an internship timeline and keeps the focus on the model rather than on front-end engineering.

5. APPLICATIONS

A churn model is valuable only when its output changes a decision. This chapter describes the practical applications of the system, moving from the retention workflow where it originated to adjacent uses in service design, financial planning and social programmes. It also states where the model should not be used, because the boundary between a useful decision aid and an inappropriate automation is an important part of professional practice.

5.1 Retention Prioritisation

The primary application is the ranking of customers by churn risk. A retention team typically has capacity to contact a few hundred customers per week. Sorting the customer base by predicted probability produces a work queue in which the expected number of true churners is much higher than in a random sample. At the selected operating point, the model catches about 79% of churners while flagging roughly 587 customers; a team that works through that list spends its effort where the risk is concentrated.

Ranking is more useful than a binary flag because it supports triage. The highest-risk customers can receive a personal call, the next tier an automated offer, and the lowest-risk customers no intervention at all. The risk bands used in the application (low below 30%, medium between 30% and 60%, high above 60%) provide a simple triage scheme that can be adjusted to the team's capacity.

5.2 Campaign Targeting and Offer Design

The factor explanation in the application helps match the intervention to the reason for risk. A customer who is flagged because of a month-to-month contract may respond to a discounted annual plan. A customer who is flagged because of high monthly charges may respond to a service review or a loyalty credit. A customer who is flagged because of fiber-optic service and no technical support may need a service-quality conversation rather than a price discount.

This is the difference between a churn score and a retention strategy. The score answers *who*; the explanation suggests *why*; the what-if simulator answers *what would change the score*. A retention manager can test a proposed intervention - for example, moving a customer from

electronic check to automatic card payment, or from month-to-month to a one-year contract - and see the estimated effect before making the offer.

5.3 Customer Segmentation and Lifetime Value

Churn probabilities combine with customer value to prioritise by expected loss, not risk alone. Expected value at risk is approximately the churn probability multiplied by the present value of future margin.

The same scores support segmentation. Customers can be grouped into risk tiers and cross-tabulated with contract type, tenure band or service mix. This produces a retention heat map that shows where risk is concentrated and whether the pattern is structural (for example, a product that consistently disappoints) or transient (for example, a billing problem affecting a cohort).

5.4 Service Quality and Product Feedback

Churn is a lagging indicator of customer experience. A model that identifies high-risk customers before they leave provides an early warning system for service problems. If a disproportionate number of high-risk customers share a particular internet service, payment method or support configuration, the operations team has a signal to investigate. In the Telco data, fiber-optic customers churn at 41.9% and electronic-check customers at 45.3%; these are not causal findings, but they are legitimate prompts for further investigation.

The model can also be used to evaluate the effect of service changes. If a new technical-support programme is introduced, the churn rate of the treated cohort can be compared with the model's expectation. This is not a substitute for a controlled experiment, but it provides a useful operational check.

5.5 Financial Planning and Forecasting

Aggregate churn probabilities can feed revenue forecasts. If the retention team knows that a certain number of high-risk customers are likely to leave, the finance team can model revenue under different retention scenarios. The probability distribution is more informative than a binary count because it allows a range of outcomes to be estimated rather than a single point forecast.

The same logic applies to capacity planning. The number of predicted churners sets the workload for the retention call centre; the expected success rate of each offer sets the number of retained customers; and the retained revenue sets the return on the campaign. A model with a well-calibrated probability is therefore a planning asset, not only a targeting tool.

5.6 Application in Social and Development Programmes

The host organisation, **BHARAT CARE LIFE LINE FOUNDATION**, works with communities and partners on welfare, skills and livelihood initiatives. The same analytical pattern - identifying participants who are at risk of disengaging and intervening early - applies to social programmes. A skills programme can use attendance and engagement data to identify learners at risk of dropping out; a livelihood programme can use participation data to identify households that may need additional support. The features and the label change, but the workflow does not.

This transferability is one of the reasons the project is relevant to the internship programme. It demonstrates that data science is not confined to commercial optimisation; the same methods can support social outcomes when they are applied with care, transparency and respect for the people represented in the data.

5.7 Where the Model Should Not Be Used

The model should not be used to deny service, set individual prices or make automated decisions without human review. It is a probabilistic estimate trained on historical data, and historical data can encode unfair patterns. If a feature such as gender or senior-citizen status is correlated with churn, using the model to allocate offers could produce discriminatory outcomes. The application is therefore framed as a decision-support tool for a human retention team, not as an autonomous decision system.

The model should also not be used outside the population on which it was trained. A model fitted to residential telecom customers may not transfer to enterprise accounts, prepaid mobile users or a different market. Before any operational use, the model should be validated on a recent, representative sample of the target population, and its performance should be monitored after deployment.

6. TECHNOLOGIES USED

The project uses a compact, well-established open-source stack. Every technology was chosen for a specific role: data manipulation, modelling, visualisation, persistence or presentation. The versions listed in this chapter are the versions used to produce the results in Chapter 10.

6.1 Python

Python is the primary implementation language. It provides readable syntax, a mature scientific ecosystem and excellent support for rapid experimentation. The project is organised into modules: `src/train.py` for data preparation, training and evaluation; `src/make_deck_assets.py` for presentation figures; `app.py` for the Streamlit application; and the report-generation modules under `src/`. All scripts are executed from the project root so that relative paths resolve consistently.

Python's dynamic typing and interactive tools make it well suited to exploratory work, while its packaging and testing ecosystem make it suitable for reproducible delivery. The project uses type hints and docstrings in the core modules to keep the code understandable.

6.2 pandas and NumPy

pandas provides the `DataFrame` abstraction used throughout the project. It handles CSV loading, column selection, grouping, aggregation, missing-value treatment and feature construction. NumPy supplies the underlying numerical arrays and is used for probability thresholds and array operations. The combination makes it possible to express data transformations concisely without writing manual loops.

pandas is also used in the application to construct a one-row DataFrame from the sidebar inputs. This ensures that the inference path receives the same column names and order as the training data, which is a common source of production errors.

6.3 scikit-learn

scikit-learn is the modelling backbone. It supplies the classifiers (`LogisticRegression`, `RandomForestClassifier`, `HistGradientBoostingClassifier`), the preprocessing transformers (`SimpleImputer`, `StandardScaler`, `OneHotEncoder`), the composition tools

(`ColumnTransformer`, `Pipeline`), the model-selection utilities (`StratifiedKFold`, `cross_val_score`, `train_test_split`) and the evaluation metrics (`roc_auc_score`, `precision_score`, `recall_score`, `f1_score`, `average_precision_score`, `confusion_matrix`, `classification_report`).

The pipeline abstraction is particularly important. It binds preprocessing and modelling into a single object that can be fitted, evaluated and serialised as a unit. This eliminates the risk of applying a different encoding at inference time and makes cross-validation honest, because the transformers are re-fitted inside each fold.

6.4 Matplotlib and Seaborn

Matplotlib is the low-level plotting library used for the architecture diagram and for charts that require precise control over layout. Seaborn is used for statistical plots such as distribution plots and heat maps. The presentation figures in this report were generated with a consistent palette and typography so that the report and the slide deck share a visual language.

Charts are saved as high-resolution PNG files rather than embedded as interactive objects. This keeps the report portable and ensures that the figures render identically on any machine.

6.5 Streamlit

Streamlit converts a Python script into an interactive web application. The application uses a sidebar for customer inputs, tabs for prediction, model performance, data insights and methodology, and custom HTML/CSS for the risk card and gauge. Streamlit's reactive execution model means that every input change triggers a re-run of the script and an updated prediction, with no explicit event handlers.

The application is intentionally thin. It does not retrain the model or recompute metrics; it loads the artifacts produced by the training script. This separation keeps the demo fast and ensures that the displayed performance numbers correspond exactly to the saved model.

6.6 joblib and JSON Artifacts

joblib serialises the fitted pipeline, including the preprocessing steps and the classifier, into `models/churn_model.joblib`. The artifact also stores the ordered feature list and the

model name. JSON files store the evaluation metrics and the EDA insights, which are read by the application and by the report generator. Plain text is used for the human-readable model report.

The use of simple, inspectable formats is deliberate. A JSON metrics file can be diffed in version control, read by any language and understood without the original code. A joblib artifact is Python-specific, but it preserves the exact fitted state of the pipeline, which is essential for reproducible inference.

6.7 Development and Documentation Tools

Development was carried out in Visual Studio Code with the Python extension, which provides debugging, linting and integrated terminals. PowerShell was used to run the training script and the Streamlit server. Microsoft Word and python-docx were used to produce this report, and python-pptx was used to produce the accompanying presentation. The project is version-controlled in a local folder structure; a Git repository can be initialised without changing the code.

6.8 Technology Stack Summary

Technology	Version	Role in the Project
Python	3.11+	Implementation language for all modules.
pandas	3.1.1	Data loading, cleaning, aggregation and feature engineering.
NumPy	2.5.4	Numerical arrays and vectorised operations.
scikit-learn	1.9.1	Preprocessing, models, cross-validation and metrics.
Matplotlib	3.11.2	Charts and the architecture diagram.
Seaborn	0.13.2	Statistical visualisations.
Streamlit	1.66.1	Interactive web application.
joblib	1.6.0	Model persistence.
python-docx	1.2.0	Programmatic report generation.
python-pptx	1.0.2	Programmatic slide generation.

Table 6.1: Technology stack and versions

The stack is intentionally conservative. Every library is widely used, well documented and available under a permissive open-source licence. This reduces the risk of maintenance problems and makes the project easy for another student or developer to continue.

7. ADVANTAGES AND DISADVANTAGES

No model is universally appropriate. A churn prediction system offers clear advantages over unaided retention work, but it also introduces costs, assumptions and risks. This chapter presents both sides honestly and describes how the project mitigates the main limitations.

7.1 Advantages

The system improves retention work in several concrete ways. The advantages below are stated in terms of what the organisation can do that it could not do before, rather than in terms of abstract model quality.

7.1.1 Prioritisation under Limited Capacity

The most immediate benefit is a ranked work queue. Retention teams cannot contact every customer, and contacting at random wastes effort on customers who would have stayed anyway. At the selected operating point the model flags 587 customers and correctly identifies 297 of the 374 churners in the test set. A random sample of the same size would be expected to contain only about 156 churners. The model therefore roughly doubles the yield of a fixed outreach budget.

7.1.2 Early Warning

Churn is observed only after the customer has left. A predictive model moves the observation point forward, giving the retention team time to intervene. This is especially important for customers on month-to-month contracts, whose departure can be abrupt. The model can be re-scored monthly, creating a rolling early-warning system.

7.1.3 Explainability

The application does not only return a score. It lists the factors that push the customer's risk up or down, and the report quantifies the global importance of each feature. This makes the output usable in a conversation with a customer or a manager, and it supports the choice of intervention. An unexplainable score is difficult to trust and difficult to act on.

7.1.4 Consistency and Reproducibility

A manual retention process depends on the judgement and memory of individual agents. Two agents may treat similar customers differently. A model applies the same logic to every

customer and produces an auditable record. The pipeline, random seeds and package versions are recorded, so the result can be reproduced and reviewed.

7.1.5 Low Operating Cost

The stack is open source, the dataset is small and the model trains in seconds on a laptop. Inference is a single `predict_proba` call. There is no GPU requirement, no managed service and no licence fee. This makes the approach accessible to small organisations and to students, which is consistent with the inclusive ethos of the internship programme.

7.1.6 Extensibility

The pipeline is modular. New features can be added to the transformer, new models can be compared in the same cross-validation loop, and the presentation layer can be replaced without retraining. The same pattern can be applied to dropout prediction, loan default, equipment failure or any other binary outcome with tabular features.

7.2 Disadvantages

The limitations fall into three groups: data limitations, modelling limitations and deployment limitations. Recognising them is part of using the model responsibly.

7.2.1 Historical and Observational Data

The model learns associations in historical data, not causation. Features such as fiber-optic internet may reflect price, service quality or competition rather than a direct cause, so associations should generate hypotheses, not prescriptions.

7.2.2 Class Imbalance and Metric Trade-offs

Only 26.5% of customers churn. The model therefore produces more false positives than a balanced problem would: at the 0.50 threshold, 290 customers are flagged who would have stayed. This is a deliberate trade-off in favour of recall, but it has a cost in outreach effort and in the risk of annoying customers who were not at risk. A different campaign economics would justify a different threshold.

7.2.3 Limited Context

The dataset contains account and service attributes but not the full context of the customer relationship: call transcripts, complaints, network outages, competitor offers and life

events are absent. The model can therefore miss sudden changes that are not reflected in the recorded features. It should be complemented by qualitative feedback from front-line staff.

7.2.4 Temporal Validity

The data is a snapshot. Customer behaviour, pricing and competition change over time, and a model trained on one period may lose accuracy in another. The project does not include a temporal validation split because the dataset does not provide reliable timestamps. In production, the model should be retrained and monitored regularly.

7.2.5 Fairness and Privacy

The dataset includes gender, age and family status. A model trained on these attributes can reproduce historical inequalities. The project does not perform a formal fairness audit, and the application is intended for decision support rather than automated action. Any operational deployment should include a fairness review, data-minimisation review and compliance check.

7.2.6 Operational Dependence

A model is only useful if its output is integrated into a workflow. If the retention team does not trust the score, cannot access it at the point of contact, or is not given the authority to act on it, the system becomes a dashboard that nobody uses. Deployment is therefore as much an organisational challenge as a technical one.

7.3 Mitigation Strategies

Risk	Mitigation in the Project
Overfitting to the test set	The test partition is created once and used only for the final evaluation; model selection uses cross-validation on the training partition.
Training-serving skew	All preprocessing is stored inside the serialised pipeline and reused at inference.
Misleading accuracy	Selection is based on ROC-AUC; recall, precision, F1 and PR-AUC are reported alongside accuracy.
Unclear model behaviour	Permutation importance and per-customer factor explanations are provided.
Class imbalance	Stratified splitting and cross-validation preserve the churn proportion in every fold.
Data quality problems	Missing values and text-encoded numbers are handled explicitly and documented.
Automation bias	The application presents a probability and risk band, not an automatic decision.
Model drift	The pipeline is reproducible and can be retrained; monitoring is proposed in Chapter 11.

Table 7.1: Risks and mitigation strategies

7.4 When to Use This Approach

The approach is appropriate when historical labelled outcomes exist, when the features are available at prediction time, and when the cost of false positives is lower than the cost of missing a genuine churner. It is less appropriate when the population is small, when the outcome is driven by sudden external shocks, or when the organisation cannot act on the output. In those cases, a simpler segmentation or a qualitative review may be more useful than a sophisticated model.

8. REQUIREMENTS

This chapter specifies the resources needed to reproduce, run and evaluate the project. The requirements are deliberately modest: the system runs on a standard student laptop and uses only open-source software. The figures below are the configurations used during development and are stated as minimums where appropriate.

8.1 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 (8th generation) or equivalent	Intel Core i5 / Ryzen 5 or better
Memory	8 GB RAM	16 GB RAM
Storage	2 GB free disk space	5 GB free solid-state storage
Display	1366 × 768	1920 × 1080
Network	Not required for local use	Required only to install packages or share the app
GPU	Not required	Not required

Table 8.1: Hardware requirements

The model is small enough to train on a CPU in a few seconds. Memory usage is dominated by the Python interpreter and the plotting libraries rather than by the dataset, which contains fewer than 10,000 rows.

8.2 Software Requirements

Software	Version	Purpose
Windows / Linux / macOS	Windows 10 or 11 (tested)	Operating system
Python	3.11 or later	Runtime for all project code
pip	Latest	Package installation
Visual Studio Code	Latest	Code editing and debugging
PowerShell	5.1 or later	Command execution
Web browser	Chrome, Edge or Firefox	Streamlit interface
Microsoft Word	2016 or later	Review of the generated report
PDF reader	Any	Review of the exported report

Table 8.2: Software requirements

8.3 Python Package Requirements

Package	Version	Use
pandas	3.1.1	Tabular data manipulation

Package	Version	Use
numpy	2.5.4	Numerical operations
scikit-learn	1.9.1	Preprocessing, models and metrics
matplotlib	3.11.2	Plotting
seaborn	0.13.2	Statistical plots
streamlit	1.66.1	Interactive application
joblib	1.6.0	Model serialisation
python-docx	1.2.0	Report generation
python-pptx	1.0.2	Presentation generation
Pillow	Latest	Image processing

Table 8.3: Python package requirements

The versions are pinned in `requirements.txt` so that a fresh environment can be recreated with `pip install -r requirements.txt`. Minor version differences are usually harmless, but the pinned versions are the ones used for the reported results.

8.4 Data Requirements

The system expects a CSV file with the following characteristics:

- One row per customer and a unique identifier column named `customerID`.
- A binary target column named `Churn` containing `Yes` or `No`.
- Twenty predictor columns covering demographics, account, services and billing.
- Numeric columns `tenure`, `MonthlyCharges` and `TotalCharges`; `TotalCharges` may contain blanks and is converted during cleaning.
- Categorical columns stored as text, with missing values allowed.

The file is stored at `data/Telco-Customer-Churn.csv`. The training script validates the presence of the target and identifier columns and fails early with a clear error if either is missing.

8.5 Functional Requirements

ID	Requirement	Acceptance Criterion
FR-01	Load and validate the raw dataset.	The script reads the CSV and confirms 7,043 rows and the required columns.
FR-02	Clean missing and malformed values.	`TotalCharges` is numeric and contains no missing values after cleaning.
FR-03	Engineer the three additional features.	`NumServices`, `AvgMonthlySpend` and `IsNewCustomer` exist for every row.
FR-04	Create a stratified train/test split.	5,634 training rows and 1,409 test rows with matching churn proportions.
FR-05	Train and compare three classifiers.	Cross-validated ROC-AUC is recorded for every model.

ID	Requirement	Acceptance Criterion
FR-06	Select and evaluate the best model.	The selected model is evaluated once on the test set and all metrics are saved.
FR-07	Explain model behaviour.	Permutation importance is computed and stored for the top features.
FR-08	Provide an interactive prediction interface.	The Streamlit app accepts customer inputs and returns a probability and risk band.
FR-09	Support what-if analysis.	Changing contract, payment or service settings updates the estimated risk.
FR-10	Persist artifacts for reuse.	The pipeline, metrics and model report are saved to disk.

Table 8.4: Functional requirements

8.6 Non-Functional Requirements

Attribute	Requirement
Usability	A non-technical user can obtain a risk score without reading documentation.
Performance	Training completes in under one minute; a prediction is returned in under one second.
Reliability	The same input and the same saved model always produce the same output.
Reproducibility	A fixed random seed and pinned package versions reproduce the reported metrics.
Maintainability	Data preparation, modelling, evaluation and presentation are separate modules.
Portability	The application runs on Windows, Linux and macOS with the same Python dependencies.
Explainability	Every prediction is accompanied by the factors that influenced it.
Privacy	The demo uses a public dataset and does not process personal data.

Table 8.5: Non-functional requirements

8.7 Installation and Execution

The project is executed from the project root with the following commands:

Code 8.1: Installation and execution commands

```
# 1. Create and activate a virtual environment (recommended)
python -m venv .venv
.venv\Scripts\activate

# 2. Install the pinned dependencies
pip install -r requirements.txt

# 3. Reproduce the training, evaluation and figures
python src/train.py

# 4. Launch the interactive application
streamlit run app.py
```

The application opens at `http://localhost:8501`. If the port is in use, an alternative port can be supplied with `--server.port`. The training script is idempotent: running it again overwrites the saved model and charts with artifacts computed from the same fixed seed.

9. METHODOLOGY AND SOURCE CODE

This chapter describes how the system was built. The methodology follows the CRISP-DM process model - business understanding, data understanding, data preparation, modelling, evaluation and deployment - adapted to the constraints of a 45-day internship. Each stage is explained, the important design choices are justified, and selected source code is reproduced so that the implementation can be reviewed without opening the repository.

9.1 Research Design

The project is an applied, quantitative study. It uses a publicly available observational dataset, a supervised learning design and a held-out evaluation. The unit of analysis is the customer; the outcome is the binary churn label; the predictors are the account, service, demographic and billing attributes. The study is cross-sectional: each customer is observed once, and the model estimates the probability associated with that profile.

CRISP-DM Phase	Activity in This Project	Output
Business understanding	Define churn, retention value and the decision the model should support.	Problem statement and objectives
Data understanding	Audit schema, missing values, distributions and class balance.	Data-quality report and EDA charts
Data preparation	Clean, engineer features, split and build the preprocessing pipeline.	Modelling table and pipeline
Modelling	Train three classifiers and compare them with cross-validation.	Cross-validated scores
Evaluation	Evaluate the selected model on the test set; interpret importance.	Metrics, curves and importance
Deployment	Package the pipeline and build the Streamlit interface.	Saved model and live application

Table 9.1: Mapping of the project to CRISP-DM

9.2 Data Acquisition

The dataset was downloaded as a single CSV file and stored in the `data` directory. The file contains 7,043 rows and 21 columns. The first column, `customerID`, is a unique identifier; the last column, `Churn`, is the target. The remaining 19 columns are predictors. No external data was merged, which keeps the study self-contained and reproducible.

The data was loaded with `pandas.read_csv`. Column names were checked against the expected schema, and the target was mapped from the strings `Yes` and `No` to the integers 1 and 0. This mapping is explicit rather than implicit, so the positive class is unambiguously the churn class.

9.3 Data Quality Audit and Cleaning

A structured audit was performed before any modelling. The audit checked row counts, duplicate identifiers, data types, missing values, category consistency and the distribution of the target. The findings are summarised in Table 9.2.

Issue	Finding	Treatment
TotalCharges type	Stored as text rather than numeric.	Converted with `pd.to_numeric(errors='coerce')`.
Missing TotalCharges	11 blank values, all with tenure = 0.	Filled with the customer's MonthlyCharges.
Identifier	customerID is unique and random.	Dropped before modelling.
Target encoding	Churn stored as Yes/No.	Mapped to 1/0.
Class balance	1,869 churners (26.5%) and 5,174 stayers (73.5%).	Stratified split and stratified cross-validation.
Category consistency	No unexpected categories or trailing spaces.	No action required.
Duplicates	No duplicate customer identifiers.	No action required.

Table 9.2: Data-quality findings and treatments

The eleven missing `TotalCharges` values are not random. They belong to customers with zero tenure, who have not yet received a bill. Imputing them with the median would be wrong because it would assign a lifetime spend to a customer who has just joined. Filling them with the monthly charge is a sensible approximation of the first bill and preserves the customer in the dataset.

Code 9.1: Data loading and cleaning

```
def load_and_clean(path: Path) -> pd.DataFrame:
    df = pd.read_csv(path)

    # TotalCharges is text with 11 blanks for tenure = 0 customers.
    df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
    df["TotalCharges"] = df["TotalCharges"].fillna(df["MonthlyCharges"])

    # Encode the target explicitly: churn = 1, stay = 0.
    df["Churn"] = df["Churn"].map({"Yes": 1, "No": 0})

    # The identifier has no predictive value.
    df = df.drop(columns=["customerID"])
    return df
```

9.4 Exploratory Data Analysis

Exploratory analysis was used to understand the data before modelling, not to select features on the test set. The analysis examined the target distribution, the relationship between churn and contract type, the tenure distribution, the relationship between churn and charges, and the churn rate across service and payment categories. The main findings are summarised below; the corresponding figures appear in Chapter 10.

The target is imbalanced: 73.5% of customers stayed and 26.5% churned. Contract type is the strongest single segmentation variable, with churn rates of 42.7%, 11.3% and 2.8% for month-to-month, one-year and two-year contracts respectively. Churners have a mean tenure of 18.0 months compared with 37.6 months for stayers, and a mean monthly charge of $74.44 compared with $61.27. Fiber-optic customers churn at 41.9%, compared with 19.0% for DSL and 7.4% for customers with no internet service. Electronic-check customers churn at 45.3%, the highest of any payment method.

These patterns are descriptive. They motivate the feature set and the later interpretation, but they are not causal claims. The modelling stage determines how much unique information each variable contributes once the others are taken into account.

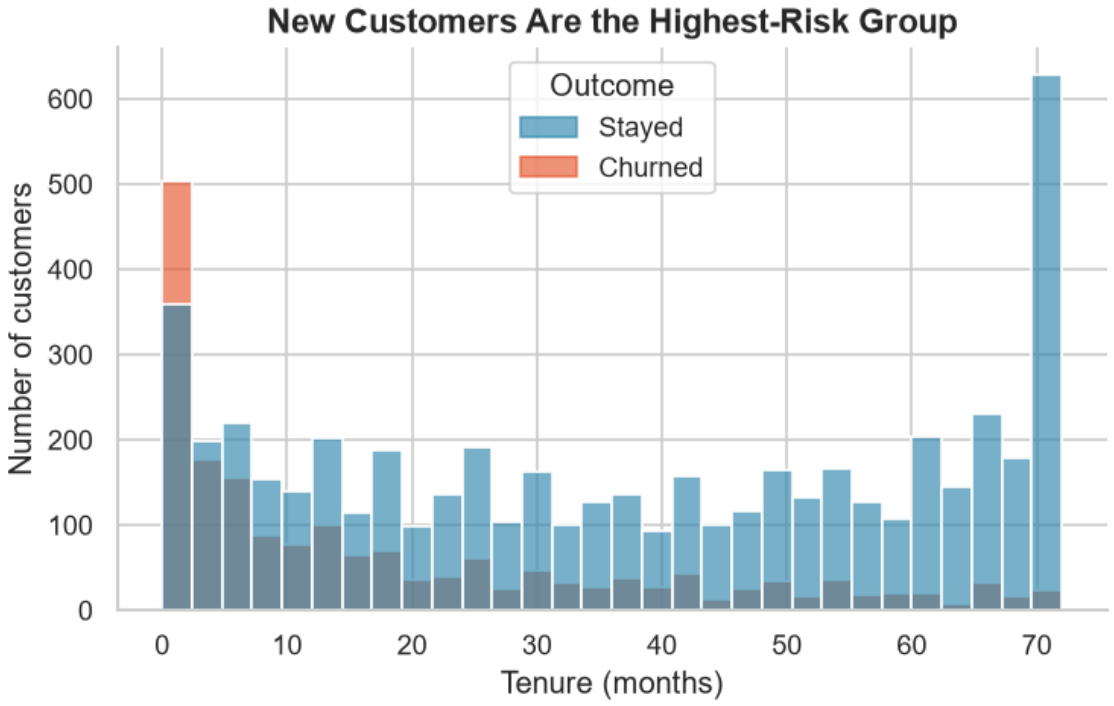


Figure 9.1: Tenure distribution by churn outcome

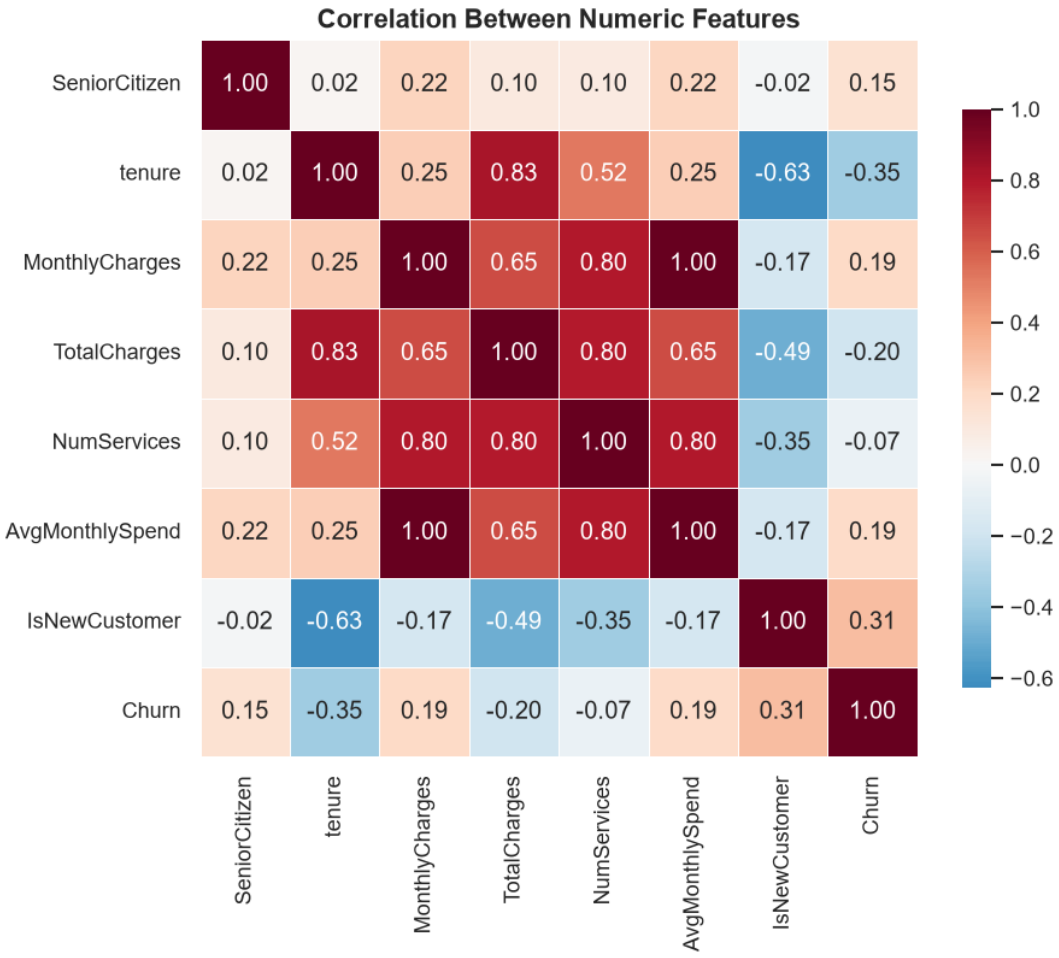


Figure 9.2: Correlation between numeric features and churn

9.5 Feature Engineering

Three features were added to the raw data. They were chosen because they are interpretable, computable at prediction time and motivated by the exploratory analysis. The first is the number of active services, which captures the breadth of the customer's relationship with the company. The second is the average monthly spend, which normalises total charges by tenure and distinguishes customers who pay a high rate from those whose total is high only because they have stayed for a long time. The third is a new-customer indicator for tenure of six months or less, which captures the high-risk early period observed in the tenure distribution.

Feature	Definition	Rationale
NumServices	Count of the eight service columns equal to `Yes`.	Customers with a broader service relationship may be more embedded, or may be paying for services they do not value.
AvgMonthlySpend	TotalCharges divided by tenure (minimum 1).	Separates high spenders from long-tenured customers whose total is large only because of time.

Feature	Definition	Rationale
IsNewCustomer	1 if tenure ≤ 6 months, otherwise 0.	The early months are the highest-risk period in the tenure distribution.

Table 9.3: Engineered features

Code 9.2: Feature engineering

```
service_cols = [
    "PhoneService", "MultipleLines", "OnlineSecurity", "OnlineBackup",
    "DeviceProtection", "TechSupport", "StreamingTV", "StreamingMovies",
]
df["NumServices"] = (df[service_cols] == "Yes").sum(axis=1)
df["AvgMonthlySpend"] = df["TotalCharges"] / df["tenure"].clip(lower=1)
df["IsNewCustomer"] = (df["tenure"] <= 6).astype(int)
```

9.6 Preprocessing Pipeline

The preprocessing pipeline is a `ColumnTransformer` with two branches. Numeric features are median-imputed and standardised. Categorical features are most-frequent-imputed and one-hot encoded. The encoder uses `handle_unknown='ignore'` so that a category not seen during training does not cause an error at inference; it is simply encoded as all zeros for that feature group.

The pipeline is fitted inside each cross-validation fold, which means that imputation and scaling parameters are learned only from the training portion of that fold. This is the correct way to prevent leakage. Fitting the scaler on the full dataset before splitting would allow information from the test set to influence the training process and would produce optimistic performance estimates.

Code 9.3: Preprocessing pipeline

```
numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipeline, NUMERIC_FEATURES),
    ("cat", categorical_pipeline, CATEGORICAL_FEATURES),
])
```

9.7 Model Selection and Training

Three classifiers were compared: Logistic Regression, Random Forest and Histogram Gradient Boosting. The three represent different inductive biases. Logistic Regression assumes a

linear relationship in the log-odds and produces a smooth, well-calibrated probability. Random Forest averages many decorrelated decision trees and captures non-linear interactions. Gradient Boosting builds trees sequentially, each one correcting the errors of its predecessor, and often achieves high accuracy on tabular data.

Model	Key Settings	Rationale
Logistic Regression	max_iter=2000, default L2 penalty, lbfgs solver	Interpretable linear baseline; stable probabilities.
Random Forest	300 trees, max_depth=None, min_samples_leaf=2, n_jobs=-1	Captures non-linearities and interactions; robust to outliers.
Histogram Gradient Boosting	learning_rate=0.08, max_iter=250, max_leaf_nodes=31	Strong tabular learner; handles mixed feature scales.

Table 9.4: Model configurations

The comparison uses stratified five-fold cross-validation on the training partition. In each fold, four parts are used to fit the pipeline and one part is used to score it. The reported score is the mean ROC-AUC across the five folds, and the standard deviation is retained as a measure of stability. The test partition is not touched during this process.

Code 9.4: Stratified cross-validation

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

for name, estimator in MODELS.items():
    pipeline = Pipeline([
        ("preprocessor", build_preprocessor()),
        ("classifier", estimator),
    ])
    scores = cross_val_score(
        pipeline, X_train, y_train,
        cv=cv, scoring="roc_auc", n_jobs=-1,
    )
    print(f"{name}: {scores.mean():.4f} +/- {scores.std():.4f}")
```

9.8 Evaluation Protocol

The model with the highest mean cross-validated ROC-AUC is selected. It is then fitted on the entire training partition and evaluated once on the test partition. The evaluation reports accuracy, precision, recall, F1, ROC-AUC and PR-AUC, together with the confusion matrix and the classification report. ROC and precision-recall curves are plotted, and permutation importance is computed on the test set to quantify the contribution of each feature.

Permutation importance is calculated by shuffling one feature at a time and measuring the resulting drop in ROC-AUC. The procedure is repeated several times per feature to obtain a mean and standard deviation. Unlike a coefficient, the measure reflects the model's actual

reliance on the feature in the context of all other features, which makes it more suitable for communicating importance to a business audience.

Code 9.5: Final evaluation

```
probs = best_model.predict_proba(X_test)[:, 1]
preds = (probs >= 0.5).astype(int)

print(classification_report(
    y_test, preds, target_names=["Stayed", "Churned"],
))
print("ROC-AUC:", roc_auc_score(y_test, probs))
print("PR-AUC :", average_precision_score(y_test, probs))
```

9.9 Threshold Selection

The reported results use a probability threshold of 0.50. This is a conventional choice that makes the results comparable with other studies, but it is not necessarily the optimal operating point for a retention campaign. If a contact costs $5 and a retained customer is worth $500, the optimal threshold is much lower than 0.50 because a false negative is far more expensive than a false positive. The application therefore exposes the probability and the risk band, allowing the retention team to apply its own cost assumptions.

The threshold should be chosen on a validation set, not on the test set. In this project, the test set is reserved for the final report; a production deployment would use a separate validation partition or a time-based holdout to tune the operating point.

9.10 Application Implementation

The Streamlit application loads the saved pipeline and feature list at start-up. The sidebar collects the customer's demographic, account, service and billing details. The application constructs a one-row DataFrame, computes the engineered features, calls `predict_proba` and renders a risk card, a gauge and a list of contributing factors. The factor logic compares the customer's values with known risk patterns from the training data, such as a month-to-month contract or electronic-check payment, and labels each factor as risk-increasing or risk-reducing.

The what-if simulator reuses the same inference function with modified contract, payment or service values. It displays the original probability, the new probability and the difference, which makes the effect of a proposed intervention concrete. The model-performance and data-insights tabs read the saved metrics and charts, so the application remains consistent with the report.

Code 9.6: Inference in the Streamlit application

```
customer = build_customer(
    gender, senior, partner, dependents, phone_service, multiple_lines,
    internet_service, online_security, online_backup, device_protection,
    tech_support, streaming_tv, streaming_movies, contract,
    paperless_billing, payment_method, tenure, monthly_charges,
)
probability = float(model.predict_proba(customer)[0, 1])
band, colour, background = risk_band(probability)
```

9.11 Reproducibility and Code Quality

The project fixes the random seed at 42 for the train/test split, the cross-validation folds and the Random Forest. Package versions are pinned in `requirements.txt`. The training script writes the model, metrics, EDA insights and figures to known paths, so the application and the report generator can consume them without manual steps. Functions are small and named after the operation they perform, and the main modules include docstrings that explain their purpose.

A limitation is that the project does not include an automated test suite or a continuous-integration pipeline. For an internship project of this scope, manual verification and a reproducible script were considered sufficient. A production version would add unit tests for the cleaning and feature-engineering functions, an integration test for the pipeline and a smoke test for the application.

10. RESULTS AND OUTPUT SCREENSHOTS

This chapter presents the results of the study. It begins with the exploratory findings, then reports the model comparison, the final test-set performance, the confusion matrix, the ranking curves and the feature importance. It concludes with screenshots of the deployed application and a discussion of the errors the model makes.

10.1 Experimental Setup

The cleaned dataset contains **7,043 customers** and 22 predictors after feature engineering. A stratified 80/20 split produced **5,634 training customers** and **1,409 test customers**. The test set contains 374 churners and 1,035 stayers, which matches the overall churn rate of 26.5%. All models were trained on the same training partition and evaluated on the same test partition with the same preprocessing pipeline.

10.2 Exploratory Findings

Figure 10.1 shows the class distribution. The imbalance is substantial: a model that always predicts *stay* would be 73.5% accurate while never identifying a churner. This is why the evaluation focuses on ROC-AUC and recall rather than accuracy.

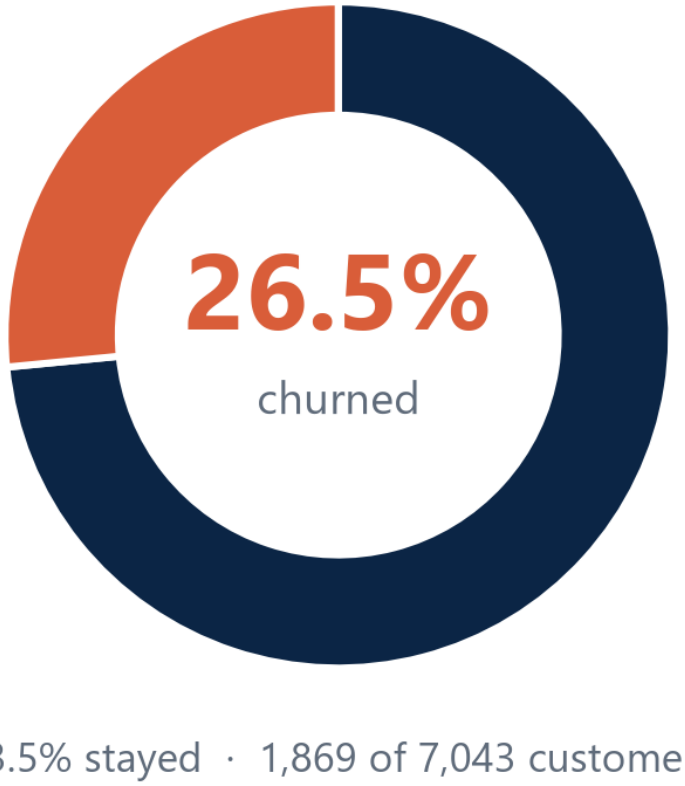


Figure 10.1: Class distribution in the dataset

Figure 10.2 shows churn by contract type. The gradient is steep: **42.7%** of month-to-month customers churn, compared with **11.3%** of one-year customers and **2.8%** of two-year customers. Contract type is therefore a powerful segmentation variable and a plausible target for retention offers, although the observational data cannot prove that upgrading a contract causes a customer to stay.

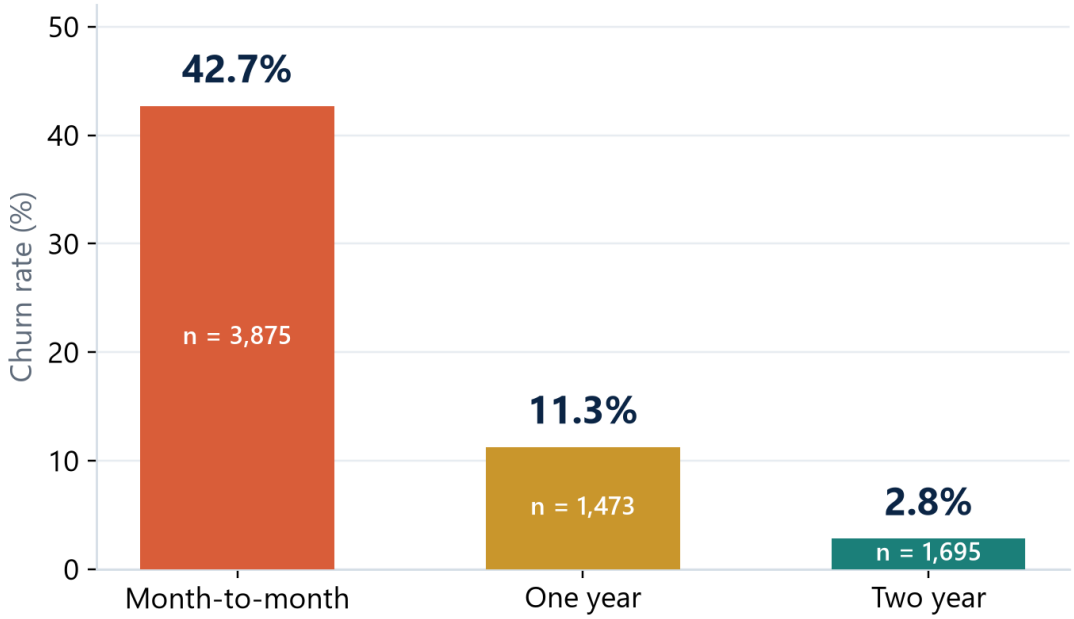


Figure 10.2: Churn rate by contract type

Figure 10.3 shows the tenure distribution. Churners are concentrated at low tenure, with a mean of 18.0 months, while stayers have a mean of 37.6 months. The shape suggests that the first year is the critical retention window and that interventions should be concentrated early in the customer lifecycle.

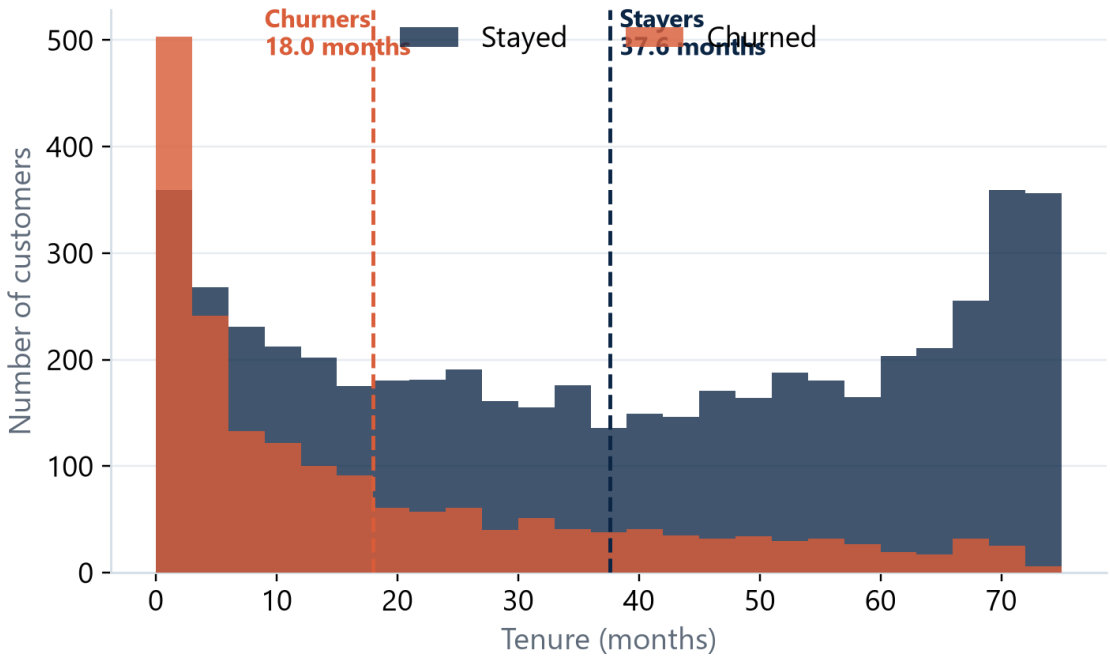


Figure 10.3: Tenure distribution for churned and retained customers

Figure 10.4 summarises four behavioural drivers. Fiber-optic customers churn at 41.9%, compared with 19.0% for DSL and 7.4% for customers without internet service. Electronic-check customers churn at 45.3%, compared with 15.2% for automatic credit card and 16.7% for automatic bank transfer. Customers without technical support or online security also show higher churn. These patterns are consistent with the hypothesis that customers who feel unsupported or who have a more transactional payment relationship are more likely to leave.

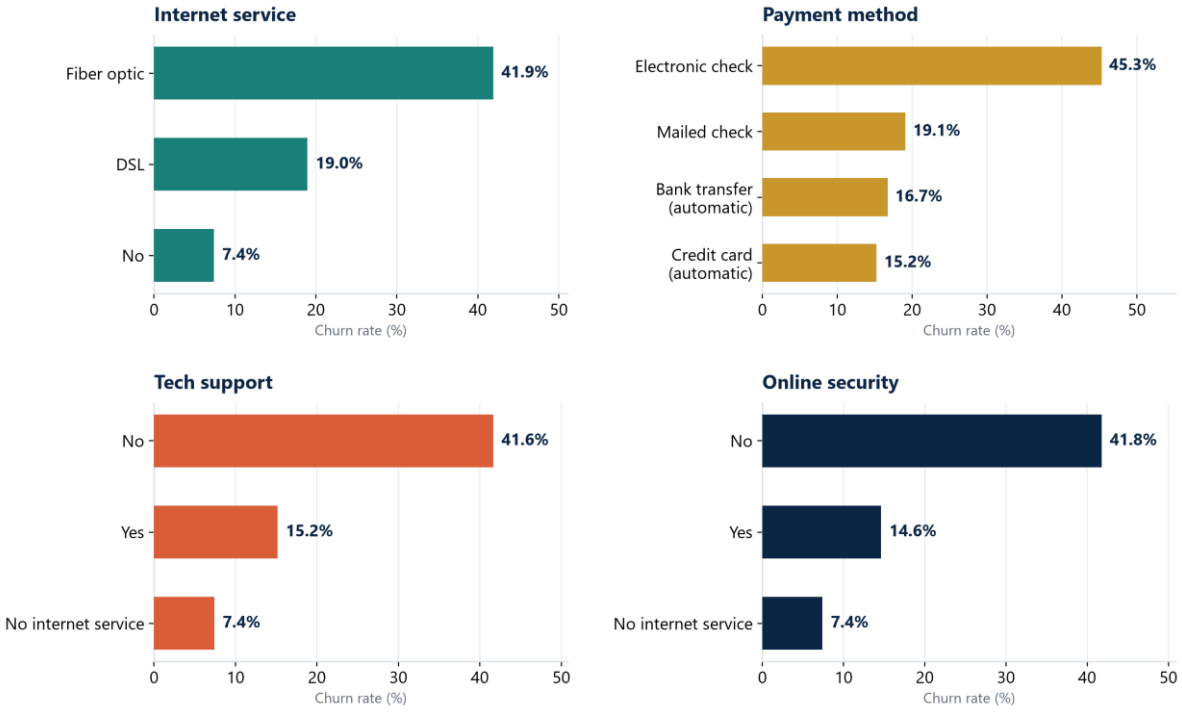


Figure 10.4: Churn rate across internet service, payment method, technical support and online security

10.3 Model Comparison

Table 10.1 reports the cross-validated ROC-AUC and the test-set metrics for the three models. Logistic Regression achieved the highest mean cross-validated ROC-AUC (0.8474) and was selected. Random Forest was extremely close (0.8457), while Gradient Boosting was lower (0.8267). The difference between the top two models is small relative to their fold-to-fold variation, but the selection rule was fixed in advance, so the result is not a post-hoc choice.

Model	CV ROC-AUC	Test ROC-AUC	Accuracy	Precision	Recall	F1
Logistic Regression	0.8474 ± 0.0110	0.8458	0.7395	0.5060	0.7941	0.6181
Random Forest	0.8457 ± 0.0093	0.8406	0.7679	0.5464	0.7406	0.6288
Gradient Boosting	0.8267 ± 0.0059	0.8255	0.7807	0.6045	0.5027	0.5489

Table 10.1: Cross-validation and test-set performance

The table reveals an important trade-off. Gradient Boosting has the highest accuracy (0.7807) and precision (0.6045) but the lowest recall (0.5027): it identifies only about half of the churners. Logistic Regression has the highest recall (0.7941) and the highest ROC-AUC on both cross-validation and the test set. For a retention use case, where missing a churner is usually more costly than contacting a customer who would have stayed, the higher recall is the more valuable property.

Random Forest is a close second on ROC-AUC and has a slightly higher F1 (0.6288) because its precision is better balanced against recall. If the retention team were strictly limited in capacity and could tolerate missing more churners, Random Forest would be a reasonable alternative. The selection of Logistic Regression is therefore justified by the stated objective - ranking quality and recall - rather than by a claim that it dominates on every metric.

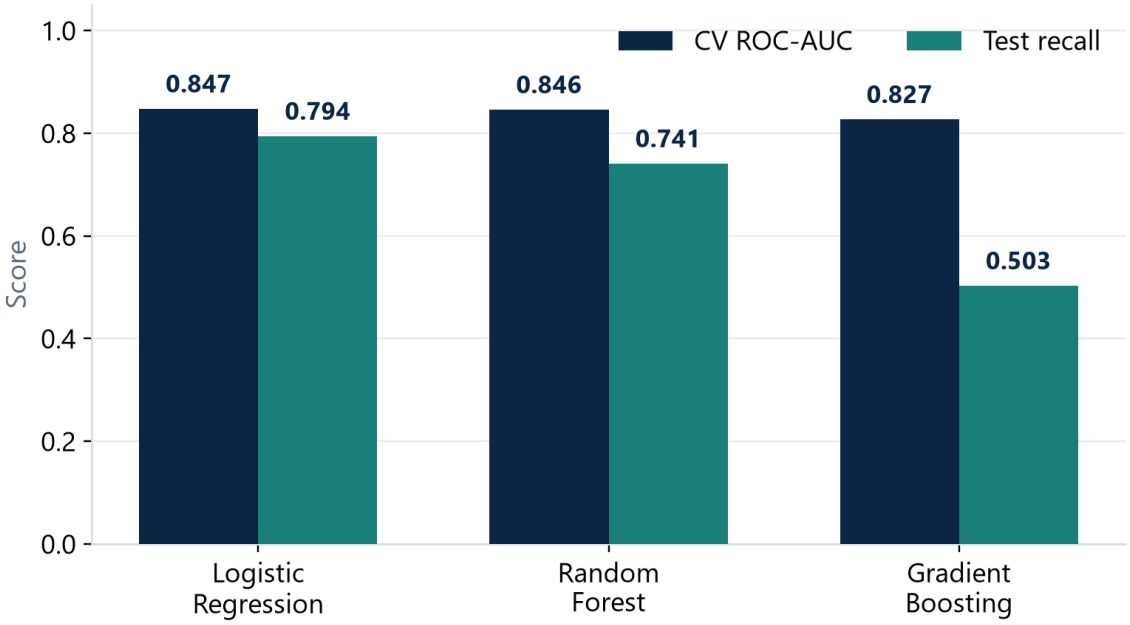


Figure 10.5: Cross-validated ROC-AUC and test recall by model

10.4 Final Model Performance

On the held-out test set, the selected Logistic Regression model achieved a **ROC-AUC of 0.846**, a **recall of 79.4%**, a **precision of 50.6%**, an **F1 of 0.618** and an **accuracy of 74.0%**. The PR-AUC is 0.653. In practical terms, the model correctly identified **297 of the 374 customers who churned** and flagged 290 customers who stayed. The full classification report is reproduced in Table 10.2.

Class	Precision	Recall	F1	Support
Stayed	0.91	0.72	0.80	1,035
Churned	0.51	0.79	0.62	374
Accuracy			0.740	1,409
Macro average	0.71	0.76	0.71	1,409
Weighted average	0.80	0.74	0.75	1,409

Table 10.2: Classification report for the selected model

The asymmetry between the classes is visible in the table. The model is good at recognising stayers (precision 0.91) and good at catching churners (recall 0.79), but the precision on churners is only 0.51. This is the expected consequence of a 0.50 threshold on an imbalanced problem: to catch four out of five churners, the model must accept that about half of the customers it flags would have stayed.

10.5 Confusion Matrix and Error Analysis

The confusion matrix in Figure 10.6 makes the error structure explicit. Of 1,035 stayers, 745 were correctly left alone and 290 were flagged. Of 374 churners, 297 were correctly flagged and 77 were missed. The false negatives are the most expensive errors for a retention team because they represent lost customers who received no intervention.

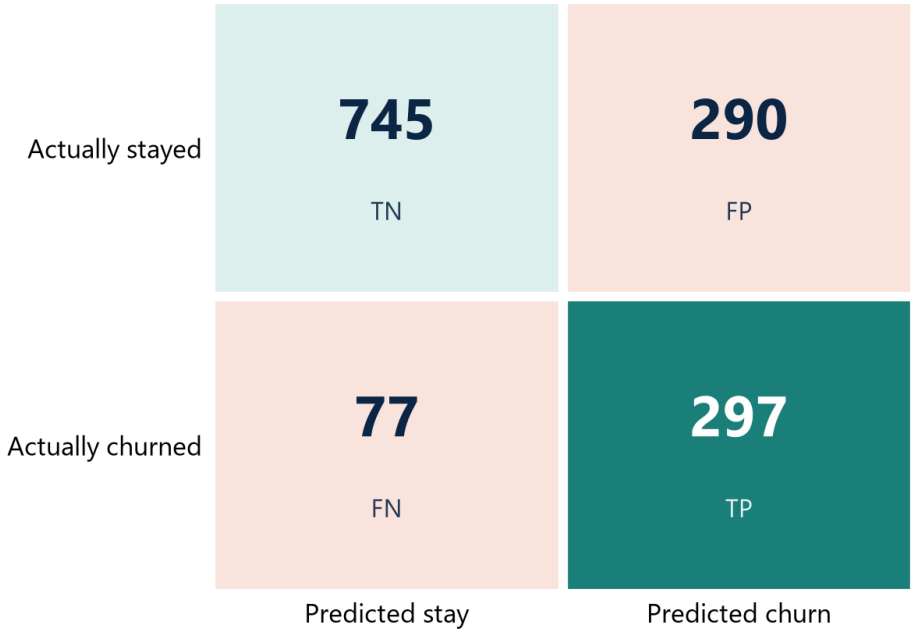


Figure 10.6: Confusion matrix on the held-out test set

The 77 missed churners are likely to be customers whose profile resembles that of stayers: long tenure, a two-year contract or a service bundle that historically correlates with retention. Their departure may be driven by factors that the dataset does not capture, such as a competitor offer, a house move or a change in household circumstances. This is a fundamental limit of a model trained on account attributes alone.

The 290 false positives are customers who were flagged but stayed. In a retention campaign they would receive an unnecessary contact or offer. This is not wasted in the same sense as a missed churner - a well-designed check-in can strengthen the relationship - but it consumes capacity and should be managed. A higher threshold would reduce false positives at the cost of more false negatives; the right balance depends on campaign economics.

10.6 Ranking Quality

Figure 10.7 shows the ROC curve. The curve rises steeply at low false-positive rates, which is the region that matters when a retention team can contact only a small fraction of the customer base. The area under the curve is 0.846, well above the random-ranking diagonal of 0.500. The PR-AUC of 0.653 is also substantially above the base churn rate of 0.265, which confirms that the model concentrates true churners near the top of the ranking.

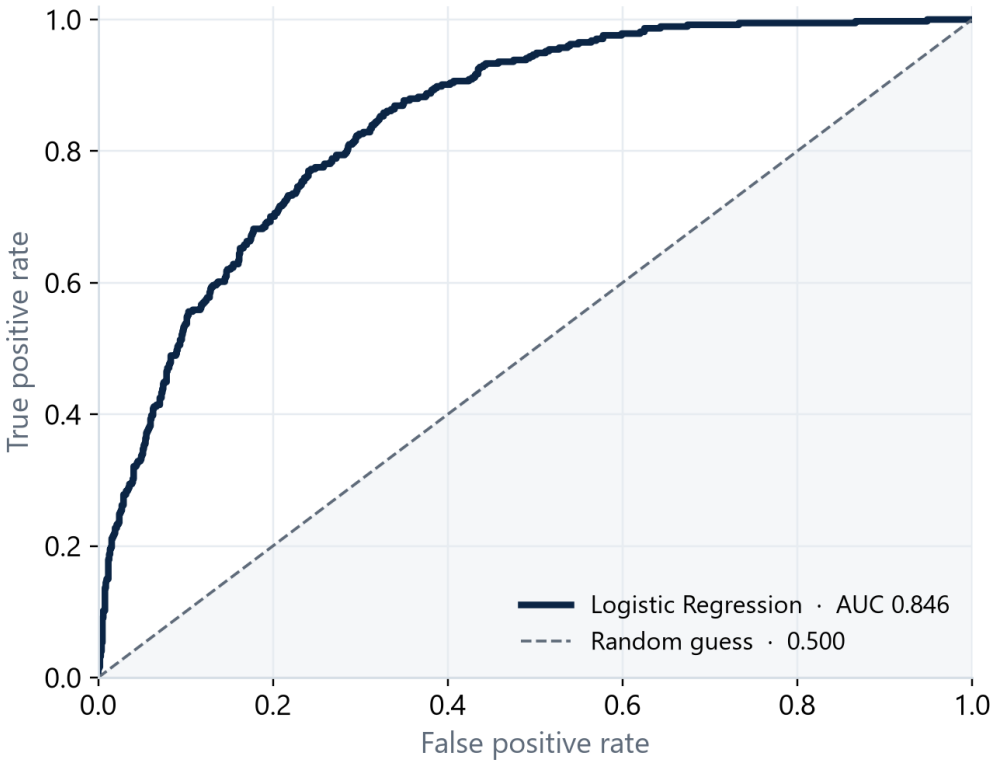


Figure 10.7: ROC curve for the selected model

The practical interpretation is a lift of approximately 1.9. If the team contacts the 587 flagged customers, it reaches 297 true churners. A random sample of 587 customers would contain about 156 churners. The model therefore nearly doubles the number of at-risk customers reached for the same outreach cost.

10.7 Feature Importance

Figure 10.8 reports permutation importance on the test set. Tenure is the most important feature, followed by contract type, internet service, monthly charges and the new-customer indicator. The ordering is consistent with the exploratory analysis and with the business intuition that the length and type of the relationship matter more than demographic attributes.

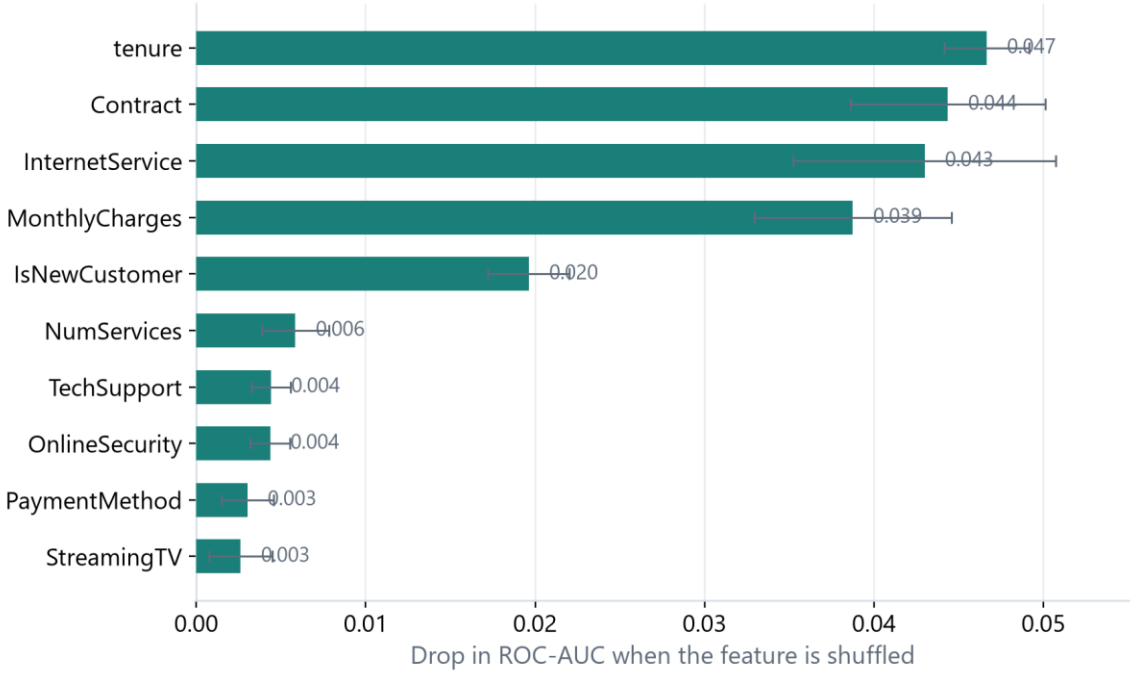


Figure 10.8: Permutation importance of the top ten features

The engineered features add modest but real value. `IsNewCustomer` is the fifth most important feature, which confirms that the early lifecycle is a distinct risk period rather than a simple linear effect of tenure. `NumServices` is sixth, suggesting that service breadth carries information beyond the individual service flags. `AvgMonthlySpend` does not appear in the top ten, which suggests that `MonthlyCharges` and `TotalCharges` already capture most of the spending signal.

Permutation importance measures reliance, not causation. A feature can be important because it is a proxy for an unobserved cause. Contract type, for example, may partly reflect self-selection: customers who expect to stay are more likely to sign a long contract. The model is therefore best used to prioritise and to generate hypotheses, not to prescribe interventions without further evidence.

10.8 Application Screenshots

The model is deployed as a Streamlit application. Figure 10.9 shows the main prediction view. The sidebar contains the customer inputs, the main panel shows the risk band, the probability gauge and the recommended actions, and the factor chips explain which attributes push the risk up or down.

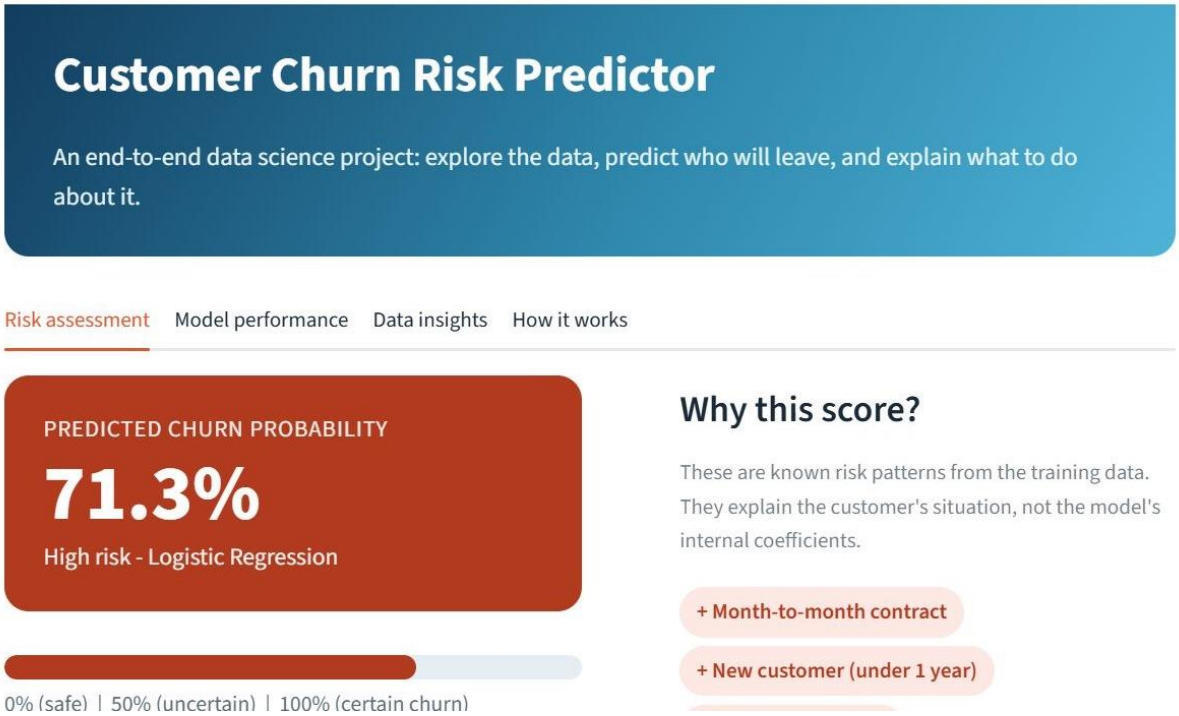


Figure 10.9: Risk assessment view of the Streamlit application

Figure 10.10 shows the what-if simulator. The user selects an alternative contract, payment method or service configuration and the application re-scores the customer. In this example, the estimated risk falls when the customer is moved to a longer contract or an automatic payment method, which illustrates how the model can support a retention conversation.

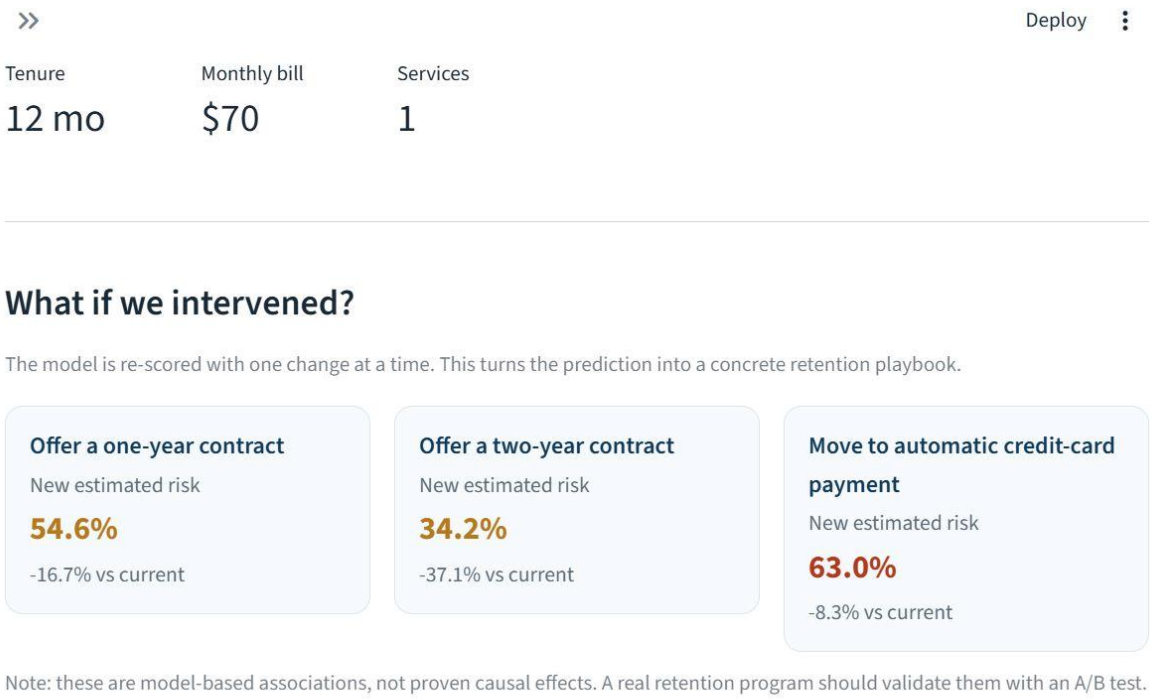


Figure 10.10: What-if retention simulator

Figure 10.11 shows the model-performance tab. It presents the headline metrics, the confusion matrix and the feature importance in a single view so that a reviewer can assess the model without leaving the application. Figure 10.12 shows the data-insights tab, which summarises the exploratory findings that motivated the feature set.

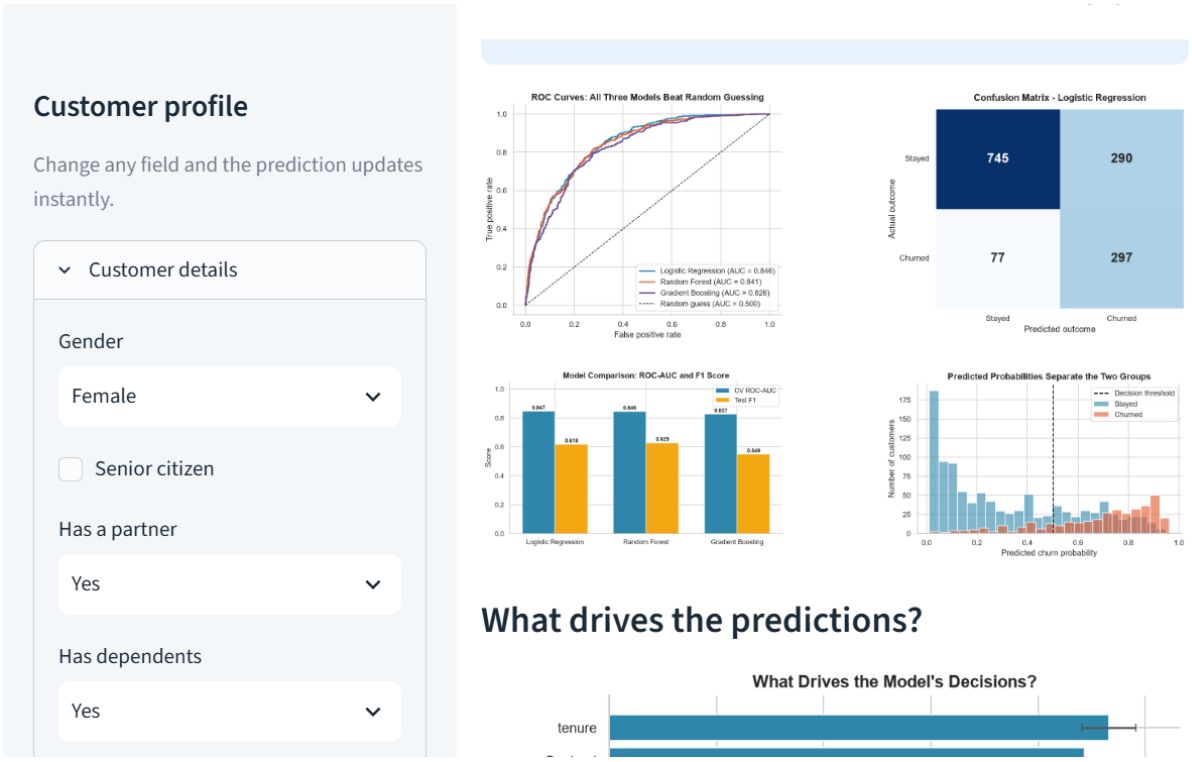


Figure 10.11: Model performance view

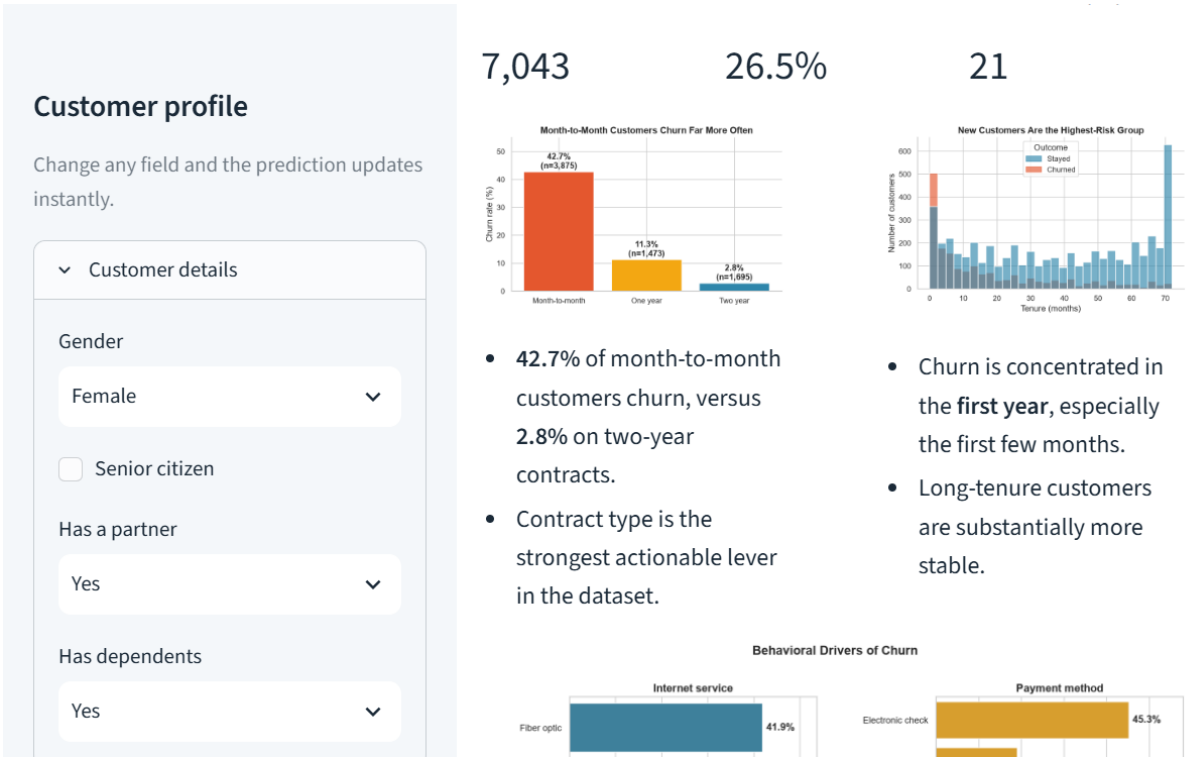


Figure 10.12: Data insights view

10.9 Comparison with a Baseline

A majority-class baseline predicts *stay* for every customer. It achieves 73.5% accuracy, zero recall, zero precision on churners and an undefined F1. The selected model has almost the same accuracy (74.0%) but a recall of 79.4% and an F1 of 0.618. The comparison shows why accuracy alone is an inadequate summary: the baseline looks respectable while providing no retention value at all.

Strategy	Accuracy	Recall (churn)	Precision (churn)	F1 (churn)
Predict everyone stays	0.735	0.000	0.000	0.000
Logistic Regression (0.50)	0.740	0.794	0.506	0.618
Random Forest (0.50)	0.768	0.741	0.546	0.629
Gradient Boosting (0.50)	0.781	0.503	0.605	0.549

Table 10.3: Comparison with baseline strategies

10.10 Discussion

The results support three conclusions. First, churn in this dataset is predictable to a useful degree: a compact linear model achieves a ROC-AUC of 0.846 and catches about four out of five churners. Second, the strongest signals are relational rather than demographic: tenure, contract type, internet service and monthly charges dominate the importance ranking. Third, the choice of model involves a genuine trade-off between recall and precision, and the right choice depends on the cost of each type of error.

The results also show the limits of a single snapshot. The model cannot explain why a particular customer leaves, and it cannot predict the effect of an intervention. It is a prioritisation tool that should be combined with human judgement, A/B testing and continuous monitoring. Within those limits, it provides a substantial improvement over random outreach and a transparent basis for a retention strategy.

11. CONCLUSION

This report has described the design, implementation and evaluation of a customer churn prediction system built during a 45-day Data Science internship. The project began with a business problem - identifying customers at risk of leaving - and ended with a deployed application that returns a risk score, explains the factors behind it and allows a what-if analysis of retention scenarios.

11.1 Summary of the Work

The IBM Telco Customer Churn dataset was cleaned, explored and enriched with three interpretable features. A leakage-free preprocessing pipeline was built with scikit-learn, and three classifiers were compared using stratified five-fold cross-validation. Logistic Regression achieved the highest cross-validated ROC-AUC (0.847) and was selected. On the held-out test set it achieved a ROC-AUC of 0.846, a recall of 79.4%, a precision of 50.6% and an F1 of 0.618, correctly identifying 297 of 374 churners. The model was deployed in a Streamlit application and documented in this report.

11.2 Key Findings

1. **Contract type is the strongest segmentation variable.** Month-to-month customers churn at 42.7%, compared with 11.3% for one-year and 2.8% for two-year contracts.
2. **Tenure is the most important model feature.** Churners have a mean tenure of 18.0 months, compared with 37.6 months for stayers, and the new-customer indicator adds independent information.
3. **Service and payment behaviour matter.** Fiber-optic customers and electronic-check customers show substantially higher churn than their alternatives.
4. **Accuracy is the wrong headline metric.** A majority-class baseline is 73.5% accurate but catches no churners; the selected model has similar accuracy and a recall of 79.4%.
5. **A simple model can be competitive.** Logistic Regression outperformed two tree-based ensembles on the selection metric while remaining interpretable and fast.
6. **Deployment changes the conversation.** The interactive application makes the model usable by a non-technical audience and turns a probability into a retention workflow.

11.3 Limitations

The study has several limitations. The dataset is a single cross-sectional snapshot, so the model cannot capture temporal drift or the effect of past interventions. The features are limited to account, service and billing attributes; call transcripts, network quality and competitor activity are absent. The model is evaluated at a single threshold and does not include a formal cost-benefit optimisation or probability calibration analysis. Fairness was considered qualitatively but not tested with group-level metrics. Finally, the application is a local demonstration rather than a production service with authentication, monitoring and audit logging.

11.4 Future Work

Several extensions would strengthen the system:

- **Temporal validation.** Train on earlier cohorts and test on later ones to estimate how performance decays over time and to support a retraining schedule.
- **Probability calibration.** Apply isotonic or Platt calibration and report reliability diagrams so that predicted probabilities can be used directly in financial planning.
- **Cost-sensitive thresholding.** Replace the fixed 0.50 threshold with an operating point derived from the cost of contact and the value of a retained customer.
- **Local explanations.** Add SHAP values to explain individual predictions more precisely than the current rule-based factor chips.
- **Fairness audit.** Evaluate performance and selection rates across gender and age groups, and document any disparities.
- **Production deployment.** Wrap the pipeline in a REST API, add authentication, logging, input validation and drift monitoring, and integrate the score into the retention team's workflow.
- **Causal evaluation.** Run a controlled experiment on a subset of flagged customers to measure the actual effect of each retention offer rather than relying on association.

11.5 Concluding Remarks

The internship provided an opportunity to work through the complete lifecycle of a data science project: understanding a business problem, preparing messy data, selecting and validating a model, interpreting the results and delivering a usable application. The technical result - a model with a ROC-AUC of 0.846 and a recall of 79.4% - is encouraging, but the more important outcome is the workflow behind it. The project demonstrates that a small, well-

documented, reproducible pipeline can turn routine customer data into a decision-support tool with measurable value.

The work was carried out under the **AVISHKARANA ANDHRA SUMMER INTERNSHIP** at **BHARAT CARE LIFE LINE FOUNDATION**, in association with **ZERO P4 Poverty**. The experience reinforced the importance of clear communication, honest evaluation and responsible use of data - lessons that will carry forward into professional practice.

12. BIBLIOGRAPHY AND REFERENCES

The following sources were consulted during the project. References are listed in the order in which they are first relevant to the report, using a consistent author-title-year style.

[1] IBM. (2019). *Telco Customer Churn* [Data set]. IBM Cognos Analytics. https://www.ibm.com/analytics (accessed July 2026).

[2] Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc.

[3] Berry, M. J. A., & Linoff, G. S. (2004). *Data mining techniques: For marketing, sales, and customer relationship management* (2nd ed.). Wiley.

[4] Breiman, L. (2001). Random forests. *Machine Learning*, *45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

[5] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics, 29*(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

[6] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

[7] McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56–61).

[8] Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., et al. (2020). Array programming with NumPy. *Nature, 585*, 357–362. https://doi.org/10.1038/s41586-020-2649-2

[9] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95.

[10] Waskom, M. L. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software, 6*(60), 3021. https://doi.org/10.21105/joss.03021

[11] Streamlit. (2026). *Streamlit documentation*. https://docs.streamlit.io (accessed July 2026).

[12] Neslin, S. A., Gupta, S., Kamakura, W., Lu, J., & Mason, C. H. (2006). Defection detection: Measuring and understanding the predictive accuracy of customer churn models. *Journal of Marketing Research, 43*(2), 204–211.

[13] Verbeke, W., Dejaeger, K., Martens, D., Hur, J., & Baesens, B. (2012). New insights into churn prediction in the telecommunication sector: A profit driven data mining approach. *European Journal of Operational Research, 218*(1), 211–229.

[14] Provost, F., & Fawcett, T. (2013). *Data science for business*. O'Reilly Media.

[15] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning* (2nd ed.). Springer.

[16] James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An introduction to statistical learning*. Springer.

[17] Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras & TensorFlow* (3rd ed.). O'Reilly Media.

[18] Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., et al. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems* (Vol. 28).

[19] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 1135–1144).

[20] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems* (Vol. 30).

[21] Microsoft. (2026). *python-docx documentation*. https://python-docx.readthedocs.io (accessed July 2026).

[22] Microsoft. (2026). *Field codes in Word*. https://support.microsoft.com (accessed July 2026).

APPENDIX A

SOURCE CODE

This appendix reproduces the source code of the project. The files are listed in the order in which they are used: data preparation and training first, then the application, then the report generators. The files in the repository are the authoritative versions; the listings here are provided for review without access to the repository.

A.1 src/train.py - Training, evaluation and figure generation

652 lines.

Code A.1: src/train.py (part 1 of 14)

```
"""
Customer Churn Prediction - end-to-end training script.


What this script does, in order:
  1. Loads the raw Telco Customer Churn dataset.
  2. Cleans it (fixes TotalCharges, converts the target to 0/1).
  3. Creates a few business-friendly engineered features.
  4. Explores the data and saves presentation-ready charts.
  5. Trains and compares three models with 5-fold cross-validation.
  6. Evaluates the best model on a held-out test set.
  7. Saves the model, metrics and charts for the Streamlit app and slides.


Run from the project root:
    python src/train.py
"""


import json
import warnings
from pathlib import Path


import joblib
import matplotlib


matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import HistGradientBoostingClassifier, RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    average_precision_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
    roc_curve,
)
from sklearn.model_selection import StratifiedKFold, cross_val_score, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

Code A.1: src/train.py (part 2 of 14)

```
warnings.filterwarnings("ignore")


RANDOM_STATE = 42
BASE_DIR = Path(__file__).resolve().parent.parent
DATA_PATH = BASE_DIR / "data" / "Telco-Customer-Churn.csv"
FIGURE_DIR = BASE_DIR / "reports" / "figures"
MODEL_DIR = BASE_DIR / "models"
REPORT_DIR = BASE_DIR / "reports"


FIGURE_DIR.mkdir(parents=True, exist_ok=True)
MODEL_DIR.mkdir(parents=True, exist_ok=True)


sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams["figure.dpi"] = 120
plt.rcParams["savefig.bbox"] = "tight"
plt.rcParams["axes.titlesize"] = 13
plt.rcParams["axes.titleweight"] = "bold"


NUMERIC_FEATURES = [
    "SeniorCitizen",
    "tenure",
    "MonthlyCharges",
    "TotalCharges",
    "NumServices",
    "AvgMonthlySpend",
    "IsNewCustomer",
]

CATEGORICAL_FEATURES = [
    "gender",
    "Partner",
    "Dependents",
    "PhoneService",
    "MultipleLines",
    "InternetService",
    "OnlineSecurity",
    "OnlineBackup",
    "DeviceProtection",
    "TechSupport",
    "StreamingTV",
    "StreamingMovies",
    "Contract",
    "PaperlessBilling",
    "PaymentMethod",
]



# ---------------------------------------------------------------------------
```

Code A.1: src/train.py (part 3 of 14)

```
# 1. Data loading and cleaning
# ---------------------------------------------------------------------------
def load_and_clean(path: Path) -> pd.DataFrame:
    df = pd.read_csv(path)


    # TotalCharges is stored as text and has 11 blank values, all for
    # brand-new customers with tenure = 0. Convert to numbers and fill
    # those blanks with their monthly charge (a sensible first bill).
    df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
    df["TotalCharges"] = df["TotalCharges"].fillna(df["MonthlyCharges"])


    # Target: Yes -> 1 (churned), No -> 0 (stayed)
    df["Churn"] = df["Churn"].map({"Yes": 1, "No": 0})


    # customerID is a random identifier with no predictive value.
    df = df.drop(columns=["customerID"])


    return df
```

```
def add_features(df: pd.DataFrame) -> pd.DataFrame:
    """Add three simple features a business audience can understand."""
    df = df.copy()


    service_cols = [
        "PhoneService",
        "MultipleLines",
        "OnlineSecurity",
        "OnlineBackup",
        "DeviceProtection",
        "TechSupport",
        "StreamingTV",
        "StreamingMovies",
    ]
    df["NumServices"] = (df[service_cols] == "Yes").sum(axis=1)
    df["AvgMonthlySpend"] = df["TotalCharges"] / df["tenure"].clip(lower=1)
    df["IsNewCustomer"] = (df["tenure"] <= 6).astype(int)


    return df



# ---------------------------------------------------------------------------
# 2. Exploratory data analysis
# ---------------------------------------------------------------------------
def save_eda_charts(df: pd.DataFrame) -> dict:
    insights = {}


    churn_rate = df["Churn"].mean()
```

Code A.1: src/train.py (part 4 of 14)

```
    insights["overall_churn_rate"] = round(float(churn_rate) * 100, 1)


    # Chart 1: class balance
    fig, ax = plt.subplots(figsize=(6, 4))
    counts = df["Churn"].value_counts().sort_index()
    bars = ax.bar(["Stayed", "Churned"], counts.values, color=["#2E86AB", "#E4572E"])
    for bar, count in zip(bars, counts.values):
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 60,
            f"{count:,}\n({count / len(df):.1%})",
            ha="center",
            va="bottom",
            fontweight="bold",
        )
    ax.set_title("Customer Churn Is Imbalanced: About 1 in 4 Leaves")
    ax.set_ylabel("Number of customers")
    ax.set_ylim(0, counts.max() * 1.22)
    sns.despine()
    fig.savefig(FIGURE_DIR / "01_churn_balance.png")
    plt.close(fig)


    # Chart 2: contract type is the strongest single signal
    contract = (
        df.groupby("Contract")["Churn"]
        .agg(["mean", "count"])
        .reindex(["Month-to-month", "One year", "Two year"])
    )
    insights["churn_by_contract"] = {
        k: round(float(v) * 100, 1) for k, v in contract["mean"].items()
    }
    fig, ax = plt.subplots(figsize=(7, 4))
    bars = ax.bar(
        contract.index,
        contract["mean"] * 100,
        color=["#E4572E", "#F3A712", "#2E86AB"],
    )
    for bar, rate, n in zip(bars, contract["mean"] * 100, contract["count"]):
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 1.2,
```

```
            f"{rate:.1f}%\n(n={n:,})",
            ha="center",
            fontweight="bold",
        )
    ax.set_title("Month-to-Month Customers Churn Far More Often")
    ax.set_ylabel("Churn rate (%)")
    ax.set_ylim(0, 55)
```

Code A.1: src/train.py (part 5 of 14)

```
    sns.despine()
    fig.savefig(FIGURE_DIR / "02_churn_by_contract.png")
    plt.close(fig)


    # Chart 3: tenure distribution
    fig, ax = plt.subplots(figsize=(7, 4))
    sns.histplot(
        data=df,
        x="tenure",
        hue="Churn",
        bins=30,
        palette={0: "#2E86AB", 1: "#E4572E"},
        alpha=0.65,
        ax=ax,
    )
    ax.set_title("New Customers Are the Highest-Risk Group")
    ax.set_xlabel("Tenure (months)")
    ax.set_ylabel("Number of customers")
    handles = [
        plt.Rectangle((0, 0), 1, 1, color="#2E86AB", alpha=0.65),
        plt.Rectangle((0, 0), 1, 1, color="#E4572E", alpha=0.65),
    ]
    ax.legend(handles, ["Stayed", "Churned"], title="Outcome", frameon=True)
    sns.despine()
    fig.savefig(FIGURE_DIR / "03_tenure_distribution.png")
    plt.close(fig)


    # Chart 4: monthly charges
    fig, ax = plt.subplots(figsize=(6.5, 4))
    sns.boxplot(
        data=df,
        x="Churn",
        y="MonthlyCharges",
        palette=["#2E86AB", "#E4572E"],
        width=0.5,
        ax=ax,
    )
    ax.set_xticklabels(["Stayed", "Churned"])
    ax.set_title("Churners Pay More per Month on Average")
    ax.set_xlabel("")
    ax.set_ylabel("Monthly charges (USD)")
    sns.despine()
    fig.savefig(FIGURE_DIR / "04_monthly_charges.png")
    plt.close(fig)


    # Chart 5: four drivers in one figure
    fig, axes = plt.subplots(2, 2, figsize=(12, 8.5))
    driver_specs = [
```

Code A.1: src/train.py (part 6 of 14)

```
        ("InternetService", "Internet service"),
        ("PaymentMethod", "Payment method"),
        ("TechSupport", "Tech support"),
        ("OnlineSecurity", "Online security"),
    ]
    colors = ["#2E86AB", "#F3A712", "#E4572E", "#6A4C93"]
    for ax, (column, title), color in zip(axes.flat, driver_specs, colors):
        rates = df.groupby(column)["Churn"].mean().sort_values(ascending=False) * 100
        sns.barplot(
            x=rates.values,
            y=rates.index,
            ax=ax,
            color=color,
            orient="h",
        )
        for i, value in enumerate(rates.values):
            ax.text(value + 0.8, i, f"{value:.1f}%", va="center", fontweight="bold")
```

```
        ax.set_title(title)
        ax.set_xlabel("Churn rate (%)")
        ax.set_ylabel("")
        ax.set_xlim(0, max(rates.values) * 1.28)
    fig.suptitle("Behavioral Drivers of Churn", fontsize=15, fontweight="bold", y=1.01)
    fig.tight_layout()
    fig.savefig(FIGURE_DIR / "05_churn_drivers.png")
    plt.close(fig)


    # Chart 6: correlation heatmap of numeric features
    numeric_cols = [
        "SeniorCitizen",
        "tenure",
        "MonthlyCharges",
        "TotalCharges",
        "NumServices",
        "AvgMonthlySpend",
        "IsNewCustomer",
        "Churn",
    ]
    fig, ax = plt.subplots(figsize=(8, 6.5))
    corr = df[numeric_cols].corr()
    sns.heatmap(
        corr,
        annot=True,
        fmt=".2f",
        cmap="RdBu_r",
        center=0,
        square=True,
        linewidths=0.5,
        cbar_kws={"shrink": 0.8},
```

Code A.1: src/train.py (part 7 of 14)

```
        ax=ax,
    )
    ax.set_title("Correlation Between Numeric Features")
    fig.savefig(FIGURE_DIR / "06_correlation_heatmap.png")
    plt.close(fig)


    # Save textual insights for the report
    insights["avg_tenure_churned"] = round(
        float(df.loc[df["Churn"] == 1, "tenure"].mean()), 1
    )
    insights["avg_tenure_stayed"] = round(
        float(df.loc[df["Churn"] == 0, "tenure"].mean()), 1
    )
    insights["avg_monthly_churned"] = round(
        float(df.loc[df["Churn"] == 1, "MonthlyCharges"].mean()), 2
    )
    insights["avg_monthly_stayed"] = round(
        float(df.loc[df["Churn"] == 0, "MonthlyCharges"].mean()), 2
    )
    insights["churn_by_internet"] = {
        k: round(float(v) * 100, 1)
        for k, v in df.groupby("InternetService")["Churn"].mean().items()
    }
    insights["churn_by_payment"] = {
        k: round(float(v) * 100, 1)
        for k, v in df.groupby("PaymentMethod")["Churn"].mean().items()
    }


    with open(REPORT_DIR / "eda_insights.json", "w", encoding="utf-8") as fh:
        json.dump(insights, fh, indent=2)


    return insights



# ---------------------------------------------------------------------------
# 3. Model building
# ---------------------------------------------------------------------------
def build_preprocessor() -> ColumnTransformer:
    numeric_pipeline = Pipeline(
        [
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
```

```
        ]
    )
    categorical_pipeline = Pipeline(
        [
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore")),
```

Code A.1: src/train.py (part 8 of 14)

```
        ]
    )
    return ColumnTransformer(
        [
            ("numeric", numeric_pipeline, NUMERIC_FEATURES),
            ("categorical", categorical_pipeline, CATEGORICAL_FEATURES),
        ]
    )



def build_models() -> dict:
    return {
        "Logistic Regression": LogisticRegression(
            max_iter=3000,
            class_weight="balanced",
            random_state=RANDOM_STATE,
        ),
        "Random Forest": RandomForestClassifier(
            n_estimators=400,
            max_depth=10,
            min_samples_leaf=3,
            class_weight="balanced_subsample",
            n_jobs=-1,
            random_state=RANDOM_STATE,
        ),
        "Gradient Boosting": HistGradientBoostingClassifier(
            max_iter=300,
            learning_rate=0.08,
            max_leaf_nodes=31,
            random_state=RANDOM_STATE,
        ),
    }



def evaluate(model, X_test, y_test) -> dict:
    probabilities = model.predict_proba(X_test)[:, 1]
    predictions = (probabilities >= 0.5).astype(int)
    return {
        "accuracy": round(float(accuracy_score(y_test, predictions)), 4),
        "precision": round(float(precision_score(y_test, predictions)), 4),
        "recall": round(float(recall_score(y_test, predictions)), 4),
        "f1": round(float(f1_score(y_test, predictions)), 4),
        "roc_auc": round(float(roc_auc_score(y_test, probabilities)), 4),
        "pr_auc": round(float(average_precision_score(y_test, probabilities)), 4),
    }



# ---------------------------------------------------------------------------
```

Code A.1: src/train.py (part 9 of 14)

```
# 4. Main pipeline
# ---------------------------------------------------------------------------
def main() -> None:
    print("[1/7] Loading and cleaning data...")
    raw = pd.read_csv(DATA_PATH)
    print(f"      Raw shape: {raw.shape[0]:,} rows x {raw.shape[1]} columns")


    df = load_and_clean(DATA_PATH)
    df = add_features(df)
    print(f"      Clean shape: {df.shape[0]:,} rows x {df.shape[1]} columns")
    print(f"      Churn rate: {df['Churn'].mean():.1%}")


    print("[2/7] Running EDA and saving charts...")
    insights = save_eda_charts(df)
```

```
    print(f"      Overall churn: {insights['overall_churn_rate']}%")
    print(f"      Month-to-month churn: {insights['churn_by_contract']['Month-to-month']}%")


    X = df.drop(columns=["Churn"])
    y = df["Churn"]


    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.20, stratify=y, random_state=RANDOM_STATE
    )
    print(f"[3/7] Train/test split: {len(X_train):,} train / {len(X_test):,} test")


    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
    models = build_models()
    results = {}
    fitted_models = {}


    print("[4/7] Training and cross-validating three models...")
    for name, estimator in models.items():
        pipeline = Pipeline(
            [("preprocessor", build_preprocessor()), ("model", estimator)]
        )
        auc_scores = cross_val_score(
            pipeline, X_train, y_train, cv=cv, scoring="roc_auc", n_jobs=-1
        )
        f1_scores = cross_val_score(
            pipeline, X_train, y_train, cv=cv, scoring="f1", n_jobs=-1
        )
        pipeline.fit(X_train, y_train)
        fitted_models[name] = pipeline
        results[name] = {
            "cv_roc_auc_mean": round(float(auc_scores.mean()), 4),
            "cv_roc_auc_std": round(float(auc_scores.std()), 4),
            "cv_f1_mean": round(float(f1_scores.mean()), 4),
            "test": evaluate(pipeline, X_test, y_test),
```

Code A.1: src/train.py (part 10 of 14)

```
        }
        print(
            f"      {name:20s} CV ROC-AUC = {auc_scores.mean():.3f} "
            f"(+/- {auc_scores.std():.3f})"
        )


    # Pick the model with the best cross-validated ROC-AUC.
    best_name = max(results, key=lambda k: results[k]["cv_roc_auc_mean"])
    best_model = fitted_models[best_name]
    print(f"[5/7] Best model: {best_name}")


    # ROC curves for all models
    fig, ax = plt.subplots(figsize=(7, 5.5))
    colors = ["#2E86AB", "#E4572E", "#6A4C93"]
    for (name, model), color in zip(fitted_models.items(), colors):
        probs = model.predict_proba(X_test)[:, 1]
        fpr, tpr, _ = roc_curve(y_test, probs)
        auc = results[name]["test"]["roc_auc"]
        ax.plot(fpr, tpr, label=f"{name} (AUC = {auc:.3f})", color=color, linewidth=2)
    ax.plot([0, 1], [0, 1], "k--", linewidth=1, label="Random guess (AUC = 0.500)")
    ax.set_xlabel("False positive rate")
    ax.set_ylabel("True positive rate")
    ax.set_title("ROC Curves: All Three Models Beat Random Guessing")
    ax.legend(loc="lower right", frameon=True)
    sns.despine()
    fig.savefig(FIGURE_DIR / "07_roc_curves.png")
    plt.close(fig)


    # Model comparison bar chart
    fig, ax = plt.subplots(figsize=(8, 4.5))
    names = list(results.keys())
    x = np.arange(len(names))
    width = 0.36
    cv_aucs = [results[n]["cv_roc_auc_mean"] for n in names]
    test_f1s = [results[n]["test"]["f1"] for n in names]
    bars1 = ax.bar(x - width / 2, cv_aucs, width, label="CV ROC-AUC", color="#2E86AB")
    bars2 = ax.bar(x + width / 2, test_f1s, width, label="Test F1", color="#F3A712")
    for bars in (bars1, bars2):
```

```
        for bar in bars:
            ax.text(
                bar.get_x() + bar.get_width() / 2,
                bar.get_height() + 0.012,
                f"{bar.get_height():.3f}",
                ha="center",
                fontsize=9,
                fontweight="bold",
            )
    ax.set_xticks(x)
```

Code A.1: src/train.py (part 11 of 14)

```
    ax.set_xticklabels(names)
    ax.set_ylim(0, 1.05)
    ax.set_ylabel("Score")
    ax.set_title("Model Comparison: ROC-AUC and F1 Score")
    ax.legend(frameon=True)
    sns.despine()
    fig.savefig(FIGURE_DIR / "08_model_comparison.png")
    plt.close(fig)


    # Confusion matrix for the best model
    probs = best_model.predict_proba(X_test)[:, 1]
    preds = (probs >= 0.5).astype(int)
    cm = confusion_matrix(y_test, preds)
    fig, ax = plt.subplots(figsize=(6.2, 5.0))
    sns.heatmap(
        cm,
        annot=True,
        fmt=",d",
        cmap="Blues",
        cbar=False,
        xticklabels=["Stayed", "Churned"],
        yticklabels=["Stayed", "Churned"],
        ax=ax,
        annot_kws={"size": 18, "fontweight": "bold"},
    )
    ax.set_title(f"Confusion Matrix - {best_name}", pad=12)
    ax.set_ylabel("Actual outcome", fontsize=12)
    ax.set_xlabel("Predicted outcome", fontsize=12)
    ax.tick_params(axis="both", labelsize=11)
    plt.setp(ax.get_yticklabels(), rotation=0, ha="right", va="center")
    fig.savefig(FIGURE_DIR / "09_confusion_matrix.png")
    plt.close(fig)


    # Permutation importance on the original features
    print("[6/7] Computing feature importance...")
    importance = permutation_importance(
        best_model,
        X_test,
        y_test,
        scoring="roc_auc",
        n_repeats=12,
        random_state=RANDOM_STATE,
        n_jobs=-1,
    )
    importance_df = pd.DataFrame(
        {
            "feature": X_test.columns,
            "importance": importance.importances_mean,
```

Code A.1: src/train.py (part 12 of 14)

```
            "std": importance.importances_std,
        }
    ).sort_values("importance", ascending=False)


    top = importance_df.head(12).iloc[::-1]
    fig, ax = plt.subplots(figsize=(8, 6))
    ax.barh(
        top["feature"],
        top["importance"],
        xerr=top["std"],
        color="#2E86AB",
        error_kw={"ecolor": "#555555", "capsize": 3},
    )
    ax.set_xlabel("Drop in ROC-AUC when the feature is shuffled")
```

```
    ax.set_title("What Drives the Model's Decisions?")
    sns.despine()
    fig.savefig(FIGURE_DIR / "10_feature_importance.png")
    plt.close(fig)


    # Probability distribution
    fig, ax = plt.subplots(figsize=(7, 4))
    sns.histplot(
        probs[y_test == 0],
        bins=25,
        color="#2E86AB",
        alpha=0.6,
        label="Stayed",
        ax=ax,
    )
    sns.histplot(
        probs[y_test == 1],
        bins=25,
        color="#E4572E",
        alpha=0.6,
        label="Churned",
        ax=ax,
    )
    ax.axvline(0.5, color="black", linestyle="--", linewidth=1.5, label="Decision threshold")
    ax.set_xlabel("Predicted churn probability")
    ax.set_ylabel("Number of customers")
    ax.set_title("Predicted Probabilities Separate the Two Groups")
    ax.legend(frameon=True)
    sns.despine()
    fig.savefig(FIGURE_DIR / "11_probability_distribution.png")
    plt.close(fig)


    # Persist the winning pipeline (preprocessing + model in one object)
    model_path = MODEL_DIR / "churn_model.joblib"
```

Code A.1: src/train.py (part 13 of 14)

```
    joblib.dump(
        {
            "model": best_model,
            "model_name": best_name,
            "features": list(X.columns),
            "metrics": results,
        },
        model_path,
    )


    summary = {
        "best_model": best_name,
        "best_model_metrics": results[best_name]["test"],
        "best_model_cv_roc_auc": results[best_name]["cv_roc_auc_mean"],
        "all_models": results,
        "dataset": {
            "rows": int(len(df)),
            "columns_after_feature_engineering": int(df.shape[1]),
            "churn_rate_pct": round(float(y.mean()) * 100, 1),
            "train_rows": int(len(X_train)),
            "test_rows": int(len(X_test)),
        },
        "top_features": importance_df.head(10).to_dict(orient="records"),
        "eda": insights,
    }
    with open(REPORT_DIR / "metrics.json", "w", encoding="utf-8") as fh:
        json.dump(summary, fh, indent=2, default=float)


    # Human-readable report
    report_lines = [
        "CUSTOMER CHURN PREDICTION - MODEL REPORT",
        "=" * 42,
        f"Dataset: Telco Customer Churn ({len(df):,} customers, {y.mean():.1%} churn)",
        f"Train/test: {len(X_train):,} / {len(X_test):,} (stratified 80/20 split)",
        f"Best model: {best_name}",
        "",
        "TEST SET METRICS (threshold = 0.50)",
    ]
    for key, value in results[best_name]["test"].items():
        report_lines.append(f"  {key:12s}: {value:.3f}")
```

```
    report_lines += [
        "",
        "ALL MODELS (cross-validated ROC-AUC)",
    ]
    for name, result in results.items():
        report_lines.append(
            f"  {name:20s}: {result['cv_roc_auc_mean']:.3f} "
            f"+/- {result['cv_roc_auc_std']:.3f}"
```

Code A.1: src/train.py (part 14 of 14)

```
        )
    report_lines += [
        "",
        "TOP 10 FEATURES (permutation importance)",
    ]
    for row in importance_df.head(10).itertuples():
        report_lines.append(f"  {row.feature:20s}: {row.importance:.4f}")
    report_lines += [
        "",
        "CLASSIFICATION REPORT (best model, test set)",
        classification_report(y_test, preds, target_names=["Stayed", "Churned"]),
    ]
    (REPORT_DIR / "model_report.txt").write_text(
        "\n".join(report_lines), encoding="utf-8"
    )


    print("[7/7] Saving artifacts...")
    print(f"      Model  -> {model_path}")
    print(f"      Charts -> {FIGURE_DIR}")
    print(f"      Report -> {REPORT_DIR / 'model_report.txt'}")
    print("")
    print("Done. Test metrics for the best model:")
    for key, value in results[best_name]["test"].items():
        print(f"  {key:12s}: {value:.3f}")



if __name__ == "__main__":
    main()
```

A.2 app.py - Streamlit application

603 lines.

Code A.2: app.py (part 1 of 13)

```
"""
Customer Churn Risk Predictor - live demo app.


Run from the project root:
    streamlit run app.py


The app loads the exact pipeline saved by src/train.py, so the preprocessing
and the model are the same objects that were evaluated in the report.
"""


import json
from pathlib import Path


import joblib
import pandas as pd
import streamlit as st


BASE_DIR = Path(__file__).resolve().parent
MODEL_PATH = BASE_DIR / "models" / "churn_model.joblib"
METRICS_PATH = BASE_DIR / "reports" / "metrics.json"
FIGURE_DIR = BASE_DIR / "reports" / "figures"


st.set_page_config(
    page_title="Churn Risk Predictor",
    page_icon=":bar_chart:",
```

```
    layout="wide",
)


# ------------------------------------------------------------------------
# Styling
# ------------------------------------------------------------------------
st.markdown(
    """
    <style>
    .block-container { padding-top: 1.6rem; padding-bottom: 3rem; max-width: 1250px; }
    .hero {
        background: linear-gradient(120deg, #123B5D 0%, #2E86AB 55%, #4FB3D9 100%);
        color: white; padding: 1.6rem 2rem; border-radius: 16px;
        margin-bottom: 1.4rem;
    }
    .hero h1 { color: white; margin: 0 0 .35rem 0; font-size: 2rem; }
    .hero p { color: #E8F4FA; margin: 0; font-size: 1.02rem; }
    .risk-card {
        border-radius: 16px; padding: 1.4rem 1.6rem; color: white;
        margin-bottom: 1rem;
    }
    .risk-card .label { font-size: .95rem; opacity: .92; letter-spacing: .04em; text-transform: uppercase; }
    .risk-card .value { font-size: 3.2rem; font-weight: 800; line-height: 1.05; margin: .2rem 0; }
```

Code A.2: app.py (part 2 of 13)

```
    .risk-card .note { font-size: .98rem; opacity: .95; }
    .gauge-track {
        width: 100%; height: 16px; background: #E6EDF3; border-radius: 999px;
        overflow: hidden; margin: .8rem 0 .3rem 0;
    }
    .gauge-fill { height: 100%; border-radius: 999px; }
    .factor {
        display: inline-block; padding: .32rem .7rem; border-radius: 999px;
        margin: .18rem .3rem .18rem 0; font-size: .88rem; font-weight: 600;
    }
    .risk-up { background: #FDE8E4; color: #B23A1E; }
    .risk-down { background: #E3F4EA; color: #1E7A46; }
    .action-card {
        background: #F7FAFC; border: 1px solid #DCE6EE; border-radius: 12px;
        padding: .85rem 1rem; margin-bottom: .6rem;
    }
    .action-card b { color: #123B5D; }
    .small-muted { color: #5C6B77; font-size: .9rem; }
    div[data-testid="stMetricValue"] { font-size: 1.65rem; }
    </style>
    """,
    unsafe_allow_html=True,
)



@st.cache_resource
def load_artifact():
    return joblib.load(MODEL_PATH)



@st.cache_data
def load_metrics():
    with open(METRICS_PATH, "r", encoding="utf-8") as fh:
        return json.load(fh)



artifact = load_artifact()
model = artifact["model"]
model_name = artifact["model_name"]
metrics = load_metrics()
feature_list = artifact["features"]


SERVICE_COLUMNS = [
    "PhoneService",
    "MultipleLines",
    "OnlineSecurity",
    "OnlineBackup",
    "DeviceProtection",
```

Code A.2: app.py (part 3 of 13)

```
    "TechSupport",
    "StreamingTV",
    "StreamingMovies",
]



def build_customer(
    gender,
    senior,
    partner,
    dependents,
    phone_service,
    multiple_lines,
    internet_service,
    online_security,
    online_backup,
    device_protection,
    tech_support,
    streaming_tv,
    streaming_movies,
    contract,
    paperless_billing,
    payment_method,
    tenure,
    monthly_charges,
):
    """Build one model-ready row, including the engineered features."""
    total_charges = round(monthly_charges * max(tenure, 1), 2)
    row = {
        "gender": gender,
        "SeniorCitizen": 1 if senior else 0,
        "Partner": partner,
        "Dependents": dependents,
        "tenure": tenure,
        "PhoneService": phone_service,
        "MultipleLines": multiple_lines,
        "InternetService": internet_service,
        "OnlineSecurity": online_security,
        "OnlineBackup": online_backup,
        "DeviceProtection": device_protection,
        "TechSupport": tech_support,
        "StreamingTV": streaming_tv,
        "StreamingMovies": streaming_movies,
        "Contract": contract,
        "PaperlessBilling": paperless_billing,
        "PaymentMethod": payment_method,
        "MonthlyCharges": monthly_charges,
        "TotalCharges": total_charges,
```

Code A.2: app.py (part 4 of 13)

```
        "NumServices": sum(
            1
            for value in [
                phone_service,
                multiple_lines,
                online_security,
                online_backup,
                device_protection,
                tech_support,
                streaming_tv,
                streaming_movies,
            ]
            if value == "Yes"
        ),
        "AvgMonthlySpend": round(total_charges / max(tenure, 1), 2),
        "IsNewCustomer": int(tenure <= 6),
    }
    return pd.DataFrame([row])[feature_list]



def predict_probability(customer: pd.DataFrame) -> float:
    return float(model.predict_proba(customer)[0, 1])
```

```
def risk_band(probability: float):
    if probability < 0.30:
        return "Low risk", "#1E7A46", "#E3F4EA"
    if probability < 0.60:
        return "Medium risk", "#B7791F", "#FDF3D9"
    return "High risk", "#B23A1E", "#FDE8E4"



def gauge_html(probability: float, color: str) -> str:
    return f"""
    <div class="gauge-track">
      <div class="gauge-fill" style="width: {probability * 100:.1f}%; background: {color};"></div>
    </div>
    <div class="small-muted">0% (safe)  |  50% (uncertain)  |  100% (certain churn)</div>
    """



# ---------------------------------------------------------------------------
# Sidebar inputs
# ---------------------------------------------------------------------------
with st.sidebar:
    st.header("Customer profile")
    st.caption("Change any field and the prediction updates instantly.")
```

Code A.2: app.py (part 5 of 13)

```
    with st.expander("Customer details", expanded=True):
        gender = st.selectbox("Gender", ["Female", "Male"])
        senior = st.checkbox("Senior citizen", value=False)
        partner = st.selectbox("Has a partner", ["Yes", "No"])
        dependents = st.selectbox("Has dependents", ["Yes", "No"])


    with st.expander("Services", expanded=False):
        phone_service = st.selectbox("Phone service", ["Yes", "No"])
        multiple_lines = st.selectbox(
            "Multiple lines", ["No", "Yes", "No phone service"]
        )
        internet_service = st.selectbox(
            "Internet service", ["Fiber optic", "DSL", "No"]
        )
        internet_options = ["No", "Yes", "No internet service"]
        online_security = st.selectbox("Online security", internet_options)
        online_backup = st.selectbox("Online backup", internet_options)
        device_protection = st.selectbox("Device protection", internet_options)
        tech_support = st.selectbox("Tech support", internet_options)
        streaming_tv = st.selectbox("Streaming TV", internet_options)
        streaming_movies = st.selectbox("Streaming movies", internet_options)
        st.caption(
            "Note: adding a service also raises the derived NumServices feature, "
            "so the net risk change combines both effects."
        )


    with st.expander("Account and billing", expanded=True):
        contract = st.selectbox(
            "Contract type", ["Month-to-month", "One year", "Two year"]
        )
        paperless_billing = st.selectbox("Paperless billing", ["Yes", "No"])
        payment_method = st.selectbox(
            "Payment method",
            [
                "Electronic check",
                "Mailed check",
                "Bank transfer (automatic)",
                "Credit card (automatic)",
            ],
        )
        tenure = st.slider("Tenure (months)", 0, 72, 12)
        monthly_charges = st.slider(
            "Monthly charges (USD)", 18.0, 120.0, 70.0, step=0.5
        )


    st.divider()
```

```
    st.caption(
        "Model: Logistic Regression trained on the IBM Telco Customer Churn "
```

Code A.2: app.py (part 6 of 13)

```
        "dataset (7,043 customers)."
    )


customer = build_customer(
    gender,
    senior,
    partner,
    dependents,
    phone_service,
    multiple_lines,
    internet_service,
    online_security,
    online_backup,
    device_protection,
    tech_support,
    streaming_tv,
    streaming_movies,
    contract,
    paperless_billing,
    payment_method,
    tenure,
    monthly_charges,
)


probability = predict_probability(customer)
band, band_color, band_bg = risk_band(probability)


# ------------------------------------------------------------------------
# Header
# ------------------------------------------------------------------------
st.markdown(
    """
    <div class="hero">
      <h1>Customer Churn Risk Predictor</h1>
      <p>An end-to-end data science project: explore the data, predict who will leave,
      and explain what to do about it.</p>
    </div>
    """,
    unsafe_allow_html=True,
)


tab_predict, tab_performance, tab_insights, tab_method = st.tabs(
    ["Risk assessment", "Model performance", "Data insights", "How it works"]
)


# ------------------------------------------------------------------------
# Tab 1: prediction
# ------------------------------------------------------------------------
```

Code A.2: app.py (part 7 of 13)

```
with tab_predict:
    left, right = st.columns([1.15, 1], gap="large")


    with left:
        st.markdown(
            f"""
            <div class="risk-card" style="background:{band_color};">
              <div class="label">Predicted churn probability</div>
              <div class="value">{probability:.1%}</div>
              <div class="note">{band} - {model_name}</div>
            </div>
            """,
            unsafe_allow_html=True,
        )
        st.markdown(gauge_html(probability, band_color), unsafe_allow_html=True)


        if probability >= 0.60:
            st.error(
                "This customer looks likely to leave. Prioritize a retention call "
```

```
                "and consider a contract or support offer."
            )
        elif probability >= 0.30:
            st.warning(
                "This customer is at moderate risk. A proactive check-in and a "
                "service review are worthwhile."
            )
        else:
            st.success(
                "This customer looks stable. Keep the experience strong and "
                "monitor for changes."
            )


        st.markdown("#### Snapshot")
        c1, c2, c3 = st.columns(3)
        c1.metric("Tenure", f"{tenure} mo")
        c2.metric("Monthly bill", f"${monthly_charges:,.0f}")
        c3.metric("Services", f"{int(customer['NumServices'].iloc[0])}")


    with right:
        st.markdown("#### Why this score?")
        st.caption(
            "These are known risk patterns from the training data. They explain "
            "the customer's situation, not the model's internal coefficients."
        )


        factors = []
        if contract == "Month-to-month":
            factors.append(("Month-to-month contract", True))
```

Code A.2: app.py (part 8 of 13)

```
        if tenure <= 12:
            factors.append(("New customer (under 1 year)", True))
        if internet_service == "Fiber optic":
            factors.append(("Fiber optic internet", True))
        if payment_method == "Electronic check":
            factors.append(("Pays by electronic check", True))
        if tech_support != "Yes" and internet_service != "No":
            factors.append(("No tech support", True))
        if online_security != "Yes" and internet_service != "No":
            factors.append(("No online security", True))
        if monthly_charges >= 80:
            factors.append(("High monthly bill", True))
        if contract in ("One year", "Two year"):
            factors.append(("Long-term contract", False))
        if tenure > 24:
            factors.append(("Loyal customer (2+ years)", False))
        if tech_support == "Yes":
            factors.append(("Has tech support", False))
        if online_security == "Yes":
            factors.append(("Has online security", False))
        if dependents == "Yes":
            factors.append(("Has dependents", False))


        if not factors:
            st.write("No strong risk or protective patterns detected.")
        else:
            chips = "".join(
                f'<span class="factor {"risk-up" if is_risk else "risk-down"}">'
                f'{"+" if is_risk else "-"} {label}</span>'
                for label, is_risk in factors
            )
            st.markdown(chips, unsafe_allow_html=True)


    st.divider()
    st.markdown("#### What if we intervened?")
    st.caption(
        "The model is re-scored with one change at a time. This turns the "
        "prediction into a concrete retention playbook."
    )


    scenarios = []
    if contract == "Month-to-month":
        scenarios.append(("Offer a one-year contract", "Contract", "One year"))
        scenarios.append(("Offer a two-year contract", "Contract", "Two year"))
```

```
    if payment_method == "Electronic check":
        scenarios.append(
            ("Move to automatic credit-card payment", "PaymentMethod", "Credit card (automatic)")
        )
```

Code A.2: app.py (part 9 of 13)

```
    if tech_support != "Yes" and internet_service != "No":
        scenarios.append(("Add tech support", "TechSupport", "Yes"))
    if online_security != "Yes" and internet_service != "No":
        scenarios.append(("Add online security", "OnlineSecurity", "Yes"))


    if not scenarios:
        st.info("This customer is already on the main protective options.")
    else:
        columns = st.columns(min(3, len(scenarios)))
        for column, (label, field, value) in zip(columns, scenarios):
            changed = customer.copy()
            changed[field] = value
            if field in ("TechSupport", "OnlineSecurity"):
                changed["NumServices"] = int(
                    (changed[SERVICE_COLUMNS] == "Yes").sum(axis=1).iloc[0]
                )
            new_probability = predict_probability(changed)
            delta = new_probability - probability
            with column:
                st.markdown(
                    f'<div class="action-card"><b>{label}</b><br>'
                    f'<span class="small-muted">New estimated risk</span><br>'
                    f'<span style="font-size:1.35rem;font-weight:700;'
                    f'color:{risk_band(new_probability)[1]};">{new_probability:.1%}</span><br>'
                    f'<span class="small-muted">{delta:+.1%} vs current</span></div>',
                    unsafe_allow_html=True,
                )


    st.caption(
        "Note: these are model-based associations, not proven causal effects. "
        "A real retention program should validate them with an A/B test."
    )


# ---------------------------------------------------------------------------
# Tab 2: model performance
# ---------------------------------------------------------------------------
with tab_performance:
    st.markdown("### How well does the model perform?")
    st.caption(
        "Evaluated on 1,409 customers the model never saw during training "
        "(20% hold-out set)."
    )


    best = metrics["best_model_metrics"]
    c1, c2, c3, c4 = st.columns(4)
    c1.metric("ROC-AUC", f"{best['roc_auc']:.3f}", help="0.5 = random, 1.0 = perfect")
    c2.metric("Recall (churners caught)", f"{best['recall']:.1%}")
    c3.metric("Precision", f"{best['precision']:.1%}")
```

Code A.2: app.py (part 10 of 13)

```
    c4.metric("F1 score", f"{best['f1']:.3f}")


    st.info(
        "**Why not just use accuracy?** 73.5% of customers stay, so a model that "
        "predicts 'everyone stays' is 73.5% accurate while catching zero churners. "
        "We optimize ROC-AUC and recall instead, because a missed churner is "
        "expensive. This model catches about 79% of customers who actually leave."
    )


    left, right = st.columns(2, gap="large")
    with left:
        st.image(str(FIGURE_DIR / "07_roc_curves.png"), use_container_width=True)
    with right:
        st.image(str(FIGURE_DIR / "09_confusion_matrix.png"), use_container_width=True)


    left, right = st.columns(2, gap="large")
    with left:
```

```
        st.image(str(FIGURE_DIR / "08_model_comparison.png"), use_container_width=True)
    with right:
        st.image(str(FIGURE_DIR / "11_probability_distribution.png"), use_container_width=True)


    st.markdown("#### What drives the predictions?")
    st.image(str(FIGURE_DIR / "10_feature_importance.png"), use_container_width=True)


    with st.expander("Full metrics for all three models"):
        rows = []
        for name, result in metrics["all_models"].items():
            rows.append(
                {
                    "Model": name,
                    "CV ROC-AUC": result["cv_roc_auc_mean"],
                    "Test ROC-AUC": result["test"]["roc_auc"],
                    "Recall": result["test"]["recall"],
                    "Precision": result["test"]["precision"],
                    "F1": result["test"]["f1"],
                    "Accuracy": result["test"]["accuracy"],
                }
            )
        st.dataframe(
            pd.DataFrame(rows).set_index("Model").style.format("{:.3f}"),
            use_container_width=True,
        )


# ---------------------------------------------------------------------------
# Tab 3: data insights
# ---------------------------------------------------------------------------
with tab_insights:
    st.markdown("### What the data told us")
```

Code A.2: app.py (part 11 of 13)

```
    st.caption(
        "7,043 telecom customers, 21 raw columns, 26.5% churn rate. "
        "These findings shaped the features and the retention recommendations."
    )


    c1, c2, c3 = st.columns(3)
    c1.metric("Customers", f"{metrics['dataset']['rows']:,}")
    c2.metric("Churn rate", f"{metrics['dataset']['churn_rate_pct']}%")
    c3.metric("Raw columns", "21")


    left, right = st.columns(2, gap="large")
    with left:
        st.image(str(FIGURE_DIR / "02_churn_by_contract.png"), use_container_width=True)
        st.markdown(
            "- **42.7%** of month-to-month customers churn, versus **2.8%** on "
            "two-year contracts.\n"
            "- Contract type is the strongest actionable lever in the dataset."
        )
    with right:
        st.image(str(FIGURE_DIR / "03_tenure_distribution.png"), use_container_width=True)
        st.markdown(
            "- Churn is concentrated in the **first year**, especially the first "
            "few months.\n"
            "- Long-tenure customers are substantially more stable."
        )


    st.image(str(FIGURE_DIR / "05_churn_drivers.png"), use_container_width=True)
    st.markdown(
        "- **Fiber optic** customers churn at 41.9%, versus 7.4% with no internet.\n"
        "- **Electronic check** payers churn at 45.3%, versus about 15-19% for "
        "automatic payment methods.\n"
        "- Customers **without tech support or online security** churn at roughly "
        "42%, versus about 15% with those services."
    )


    left, right = st.columns(2, gap="large")
    with left:
        st.image(str(FIGURE_DIR / "04_monthly_charges.png"), use_container_width=True)
    with right:
        st.image(str(FIGURE_DIR / "01_churn_balance.png"), use_container_width=True)
```

```
    with st.expander("Correlation heatmap and extra charts"):
        st.image(str(FIGURE_DIR / "06_correlation_heatmap.png"), use_container_width=True)


# ---------------------------------------------------------------------------
# Tab 4: method
# ---------------------------------------------------------------------------
with tab_method:
```

Code A.2: app.py (part 12 of 13)

```
    st.markdown("### The pipeline, in plain English")


    steps = [
        ("1. Clean", "Fixed 11 blank TotalCharges values, converted Yes/No to 0/1, dropped the customer ID."),
        ("2. Explore", "Compared churn across contract, tenure, services, billing and charges."),
        ("3. Engineer", "Added number of services, average monthly spend and a new-customer flag."),
        ("4. Split", "80% training / 20% test, stratified so both sets keep the same churn rate."),
        ("5. Train", "Compared Logistic Regression, Random Forest and Gradient Boosting with 5-fold CV."),
        ("6. Evaluate", "Chose the best ROC-AUC, then checked recall, precision, F1 and the confusion matrix."),
        ("7. Deploy", "Saved preprocessing + model together, and served it in this Streamlit app."),
    ]
    for title, description in steps:
        st.markdown(f"**{title}.** {description}")


    st.markdown("#### Features used by the model")
    left, right = st.columns(2, gap="large")
    with left:
        st.markdown(
            "**Numeric (7)**\n"
            "- `SeniorCitizen`\n"
            "- `tenure`\n"
            "- `MonthlyCharges`\n"
            "- `TotalCharges`\n"
            "- `NumServices` (engineered)\n"
            "- `AvgMonthlySpend` (engineered)\n"
            "- `IsNewCustomer` (engineered)"
        )
    with right:
        st.markdown(
            "**Categorical (15)**\n"
            "- `gender`, `Partner`, `Dependents`\n"
            "- `PhoneService`, `MultipleLines`, `InternetService`\n"
            "- `OnlineSecurity`, `OnlineBackup`, `DeviceProtection`, `TechSupport`\n"
            "- `StreamingTV`, `StreamingMovies`\n"
            "- `Contract`, `PaperlessBilling`, `PaymentMethod`"
        )
    st.caption(
        "Target: Churn (1 = left, 0 = stayed). Categorical fields are one-hot encoded; "
        "numeric fields are scaled. customerID was dropped."
    )
    st.info(
        "**Feature interaction to know about:** NumServices is derived from the service "
        "toggles. Adding DeviceProtection changes both DeviceProtection and NumServices, "
        "so the net effect can differ from the service's isolated effect. On its own, "
        "DeviceProtection is slightly protective in this model (odds ratio about 0.95); "
        "the extra service count is what can nudge the score up."
    )
```

Code A.2: app.py (part 13 of 13)

```
    st.divider()
    st.markdown("#### Why Logistic Regression?")
    st.markdown(
        "It won the cross-validated ROC-AUC comparison (0.847), tied with Random "
        "Forest (0.846) and beat Gradient Boosting (0.827). It is also simple, "
        "fast, well-calibrated and easy to explain to a business audience - a rare "
        "case where the most interpretable model is also the most accurate."
    )


    st.markdown("#### Limitations")
    st.markdown(
        "- The dataset is from one telecom company and is not recent, so results "
        "may not transfer directly.\n"
        "- The model finds associations, not causes. Retention offers should be "
        "tested experimentally.\n"
```

```
        "- The 0.50 threshold is a starting point; the business should choose a "
        "threshold based on the cost of a missed churner versus a wasted offer.\n"
        "- No customer lifetime value or competitor data is included."
    )

    st.markdown("#### Next steps")
    st.markdown(
        "- Add SHAP explanations for per-customer reasoning.\n"
        "- Track model performance over time and retrain as behavior changes.\n"
        "- Run A/B tests on the retention offers suggested by the what-if tool.\n"
        "- Add customer lifetime value so retention spending is prioritized by profit."
    )
```

A.3 src/report_config.py - Shared report configuration

77 lines.

Code A.3: src/report_config.py (part 1 of 2)

```
"""Shared configuration and data for the internship report."""


import json
from pathlib import Path


BASE = Path(__file__).resolve().parent.parent
REPORTS = BASE / "reports"
ASSETS = REPORTS / "assets"
FIGURES = REPORTS / "figures"
DECK_FIGURES = REPORTS / "deck_figures"
APP_SHOTS = REPORTS / "app_screens"


DOCX_PATH = REPORTS / "Customer_Churn_Prediction_Internship_Report.docx"
PDF_PATH = REPORTS / "Customer_Churn_Prediction_Internship_Report.pdf"
TOC_PATH = REPORTS / "toc_pages.json"


with open(REPORTS / "metrics.json", "r", encoding="utf-8") as fh:
    METRICS = json.load(fh)


with open(REPORTS / "eda_insights.json", "r", encoding="utf-8") as fh:
    EDA = json.load(fh)


STUDENT = {
    "name": "P. Yugendra",
    "reg_no": "239Y1A05C2",
    "degree": "BACHELOR OF TECHNOLOGY",
    "branch": "COMPUTER SCIENCE AND ENGINEERING",
    "college": "K.S.R.M. COLLEGE OF ENGINEERING",
    "college_short": "K.S.R.M. College of Engineering",
    "department": "DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING",
    "address": "KADAPA, ANDHRA PRADESH",
    "academic_year": "2026 – 2027",
    "programme": "AVISHKARANA ANDHRA SUMMER INTERNSHIP",
    "organisation": "BHARAT CARE LIFE LINE FOUNDATION",
    "partner": "in association with ZERO P4 POVERTY",
    "duration": "45-Day Internship, Summer 2026",
    "issue_date": "15 August 2026",
    "coordinator": "A. Vamsi Reddy",
    "coordinator_role": "Coordinator, Bharat Care Life Line Foundation",
    "hod": "Dr. P. V. Kishore",
    "hod_role": "Associate Professor & HoD, CSE",
    "college_coordinator": "Dr. V. Lokeswara Reddy",
    "college_coordinator_role": "Professor & Internship Coordinator, CSE",
    "project_title": "Customer Churn Prediction Using Machine Learning",
    "project_short": "Customer Churn Prediction",
}


# Chapter titles used in the index and for page detection.
```

Code A.3: src/report_config.py (part 2 of 2)

```
CHAPTERS = [
    (1, "INTRODUCTION"),
```

```
    (2, "HISTORY"),
    (3, "DEFINITION"),
    (4, "SYSTEM ARCHITECTURE"),
    (5, "APPLICATIONS"),
    (6, "TECHNOLOGIES USED"),
    (7, "ADVANTAGES AND DISADVANTAGES"),
    (8, "REQUIREMENTS"),
    (9, "METHODOLOGY AND SOURCE CODE"),
    (10, "RESULTS AND OUTPUT SCREENSHOTS"),
    (11, "CONCLUSION"),
    (12, "BIBLIOGRAPHY AND REFERENCES"),
]

INDEX_TITLES = {
    1: "Introduction",
    2: "History",
    3: "Definition",
    4: "System Architecture",
    5: "Applications",
    6: "Technologies Used",
    7: "Advantages and Disadvantages",
    8: "Requirements",
    9: "Methodology and Source Code",
    10: "Results and Output Screenshots",
    11: "Conclusion",
    12: "Bibliography and References",
}
```